

Anime Recommendation System

Nishchal Gaur, Yogesh Kumar, Sarthak Raj, Niket Agarwal
Group 56

ML-Endsem Project



INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY
DELHI



Motivation



Given the thousands of available titles, the vast diversity of anime makes finding new series that match personal preferences challenging.

This challenge inspired the creation of an anime recommendation system. The project aims to solve the common dilemma

“What do I watch next?”

, reducing time spent searching for recommendations and more on enjoying anime.



The goal is to provide a more effective suggestion process, enhance user experience, and highlight potentially overlooked content that aligns with viewers' tastes.

Literature review



The literature explains a "hybrid approach to building an anime recommendation system."

It is a combination of two popular methods discussed as

- a. **Content-based filtering**: This method suggests items based on the characteristics or features of the items (e.g., genre, tags) and the user's preferences.
- b. **Collaborative filtering**: This method relies on user interactions (e.g., ratings) and recommends items based on similar users' liking.

Purpose of the Research Paper: The paper describes using a hybrid method to build a system that recommends anime, combining both content-based and collaborative filtering approaches to reduce their respective drawbacks, such as:

- a. Content-based filtering struggles when item attributes are inaccurate.
- b. Due to insufficient data, collaborative filtering may perform poorly for new users or items.

Dataset: They used the MyAnimeList dataset, available on Kaggle, for their system.

Hybrid Filtering Techniques: Several hybrid filtering techniques are explained:

- a. **Weighted Hybridization**: A weighted sum of predictions for both methods.
- b. **Switching Hybridization**: Switching between methods based on performance.
- c. **Mixed Hybridization**: Using multiple methods simultaneously to handle cold start problems (e.g., new items or users).
- d. **Cascade Hybridization**: One method generates an initial ranking, and another refines it.

The conclusion is that the hybrid method was more efficient and accurate in recommending anime than individual methods. This system helps users find anime based on what they've liked before and what similar users have liked, combining item attributes and user interactions.

RESEARCH PAPER LINK: https://www.ijirset.com/upload/2019/november/17_Hybrid_NEW.PDF

Literature review



It describes an Anime Recommendation System built using various techniques and models. Here are some key points:

Purpose: The system aims to recommend anime to users based on their preferences. It integrates multiple techniques like collaborative filtering, content-based filtering, and deep learning methods (Autoencoders, RNN, LSTM).

Challenges: The system addresses several common issues in recommendation systems:

- a. **Cold start problem:** Difficulty recommending items for new users or new items.
- b. **Sparsity problem:** Users may interact with only a few items, making it hard to form recommendations.
- c. **Diversity and bias:** The tendency of systems to recommend popular items repeatedly leads to a need for more variety.
- d. **Synonym issue:** Differences in anime names (e.g., "Kimetsu no Yaiba" vs. "Demon Slayer") can confuse the system.

Models and Techniques Used:

- a. **Content-based filtering:** Recommends anime based on its attributes (genres, producers, etc.) and the user's history.
- b. **Collaborative filtering:** Uses user-item interactions to suggest anime based on similar users' preferences.
- c. **RNN with LSTM and GRU:** Bidirectional Recurrent Neural Networks (BiRNN) with Long Short-Term Memory (LSTM) or Gated Recurrent Units (GRU) process user data sequentially and recommend anime based on favorite anime lists and tags.
- d. **Attention Mechanism:** Focuses on relevant parts of the input data to improve recommendations.
- e. **Ensemble Learning:** Combines the outputs of multiple models to enhance performance.

Conclusion: The system combines several advanced techniques to improve accuracy, particularly for new users and anime. It recommends anime based on the most frequent recommendations from multiple models, providing a robust and diverse output. This recommendation engine offers a more precise and efficient approach to anime recommendations than traditional methods.

RESEARCH PAPER LINK: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4491482

Dataset description



Introduction

1. We took the dataset containing information about **Anime and Otaku** who watch it.
2. This dataset aims to be a **representative sample of the internet otaku community** for demographics analysis and trends inside this group. It contains information about users (gender, location, birth date, etc.), anime (airing date, genres, producer...), and anime lists.
3. Users in MyAnimeList can add anime to their lists and mark it as a plan to watch, complete, watch, or drop, and they can also rate it by scoring 1-10.
4. **Anime_cleaned.csv** contains the list of anime, with title, title synonyms, genre, studio, licensor, producer, duration, rating, score, airing date, episodes, source (manga, light novel, etc.), and many other important data about individual anime providing sufficient information about trends in time about essential aspects of anime. Rank is in float format in CSV but contains only integer values. This is due to NaN values and their representation in pandas.
5. **Users_cleaned.csv** contains information about users who watch anime, namely a username, registration date (join_date), last online date, birth date, gender, location, and lots of aggregated values from their anime lists.
6. **Animelists_cleaned.csv** contains anime lists of all users. Per each record, here is the username, anime ID, score, status, and timestamp when this record was last updated.

Dataset Summary

1. Unique Users: 302,675
2. Users with Demographic Data: 302,573
3. Anime List Records: 80,076,112
4. Records with Ratings: 46,358,322
5. Unique Anime: 14,478

Dataset description



Data Cleaning

Missing Values:

If there are a lot of null values in a particular feature, then we have to drop that feature for ease of handling of datasets.

- Dropping Not useful Columns: Dropping all the unnecessary columns and columns containing many null values from all three datasets.
- Dropping Rows with Null Values.

Removing Duplicate Values:

First, check and then remove if there are any duplicate values in the df_anime and df_users data frame. conclusion: There are no Duplicate entries(anime,username) Present in df_anime and df_users

Cleaning my_last_updated column in df_animelists data frame:

As can be seen, the my_last_updated column contains the last updated date and time. So, to make data handling easy, we will only use the last update date and remove the last updated time part from the my_last_updated column in the df_animelists data frame.

Cleaning birth_date column in df_users data frame:

As can be seen, the birth_date column also contains the date and time. But we only want the date part from the birth_date column in the df_users data frame, so it can make data handling easy.

Dataset description



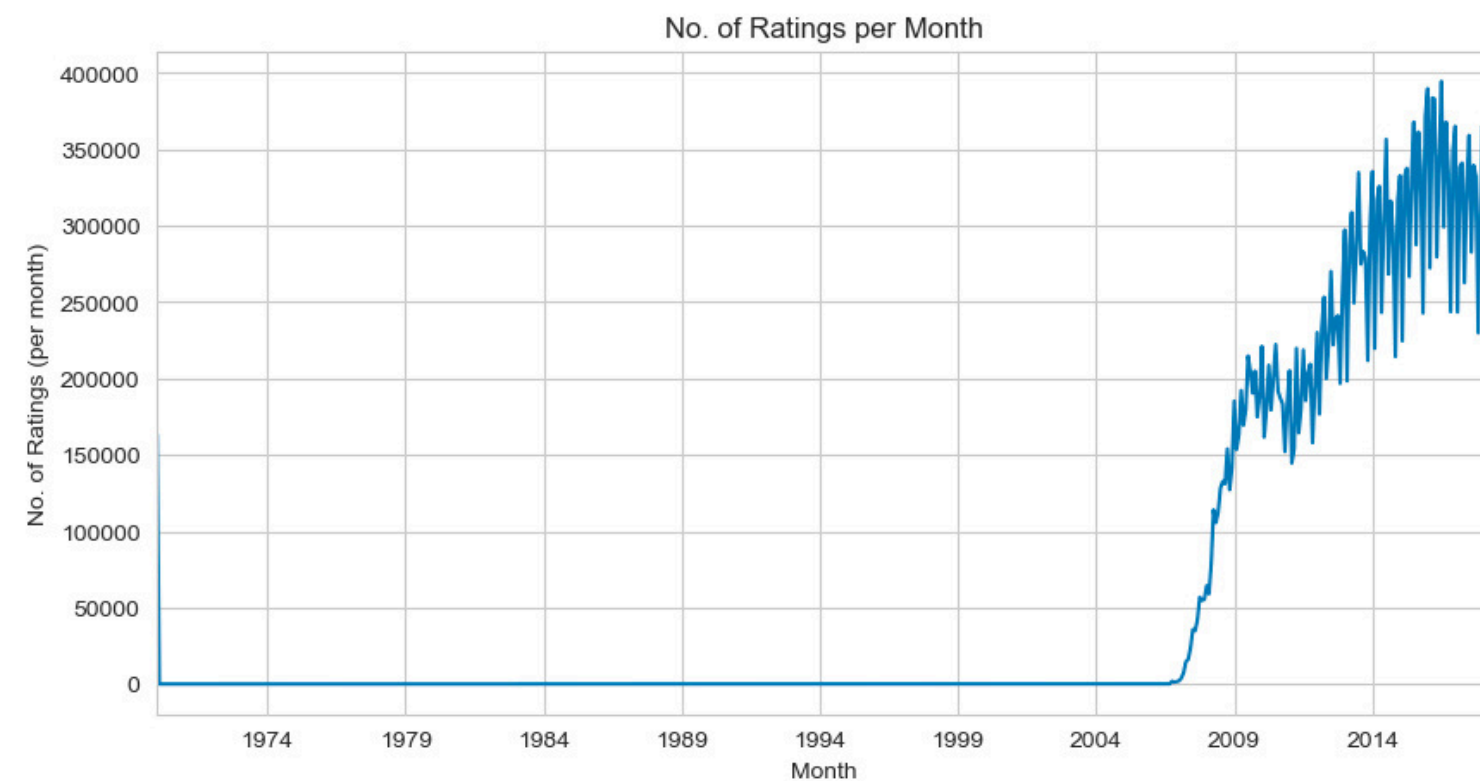
Cleaning rows having outlier value of my_last_updated column in df_animelists:

We are cleaning the my_last_updated column in df_animelists because it may contain dates that are not realistic.

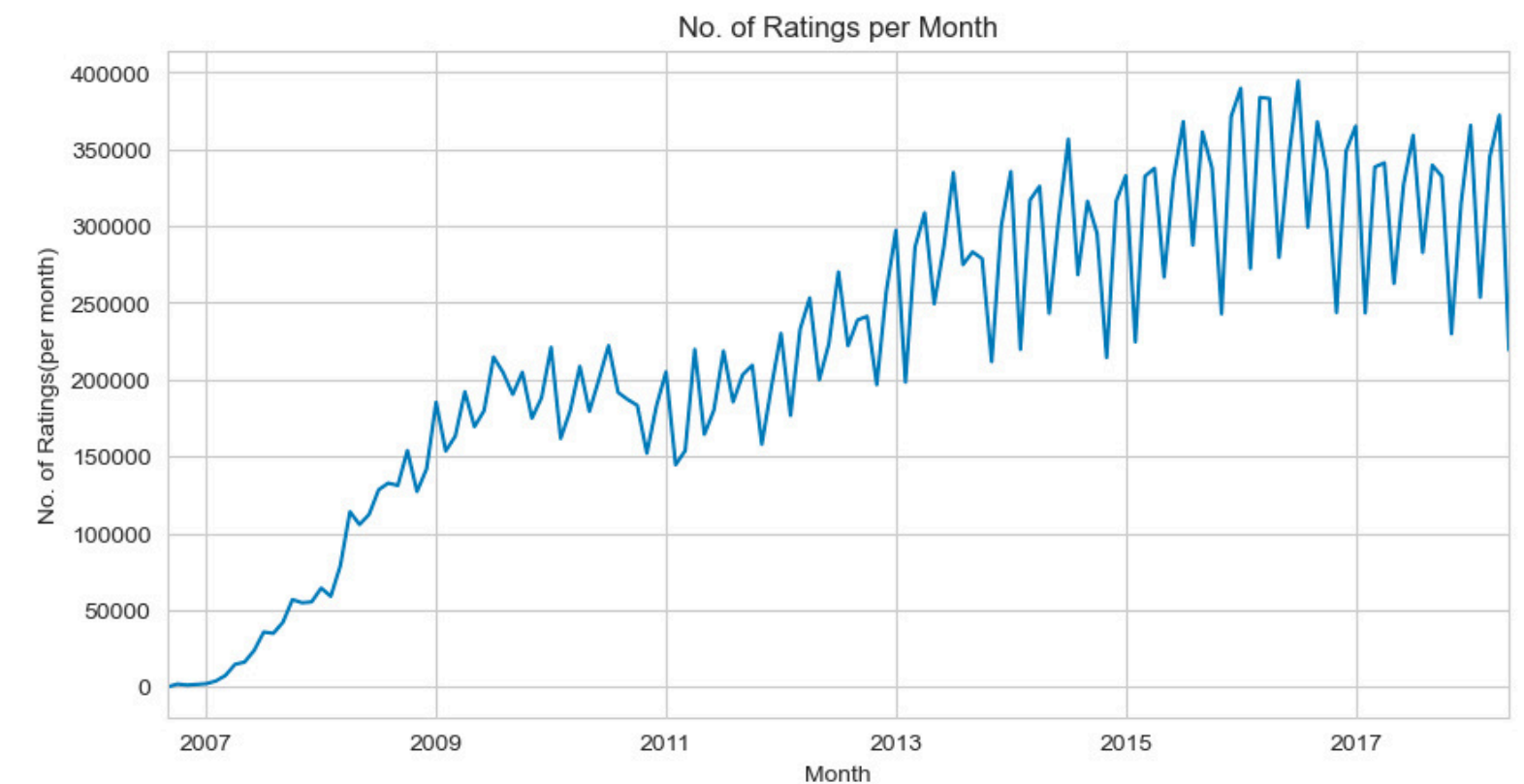
It can be observed that the my_last_updated column contains the date that goes back to 1970-01-01, which is not possible as the website MyAnimeList itself starts from 2004-11-05, so having a my_last_updated as 1970-01-01 or any date before 2004-11-05 is not possible and it needs to be removed.

So, let's plot the distribution of my_last_updated column in the df_animelists data frame. (GRAPH 1)

Observation: Observing the graph, it seems that my_last_updated column only has the 1970-01-01 date as an outlier, and no other outlier date is between 1970-01-01 and 2004-11-05.



Before Data Cleaning

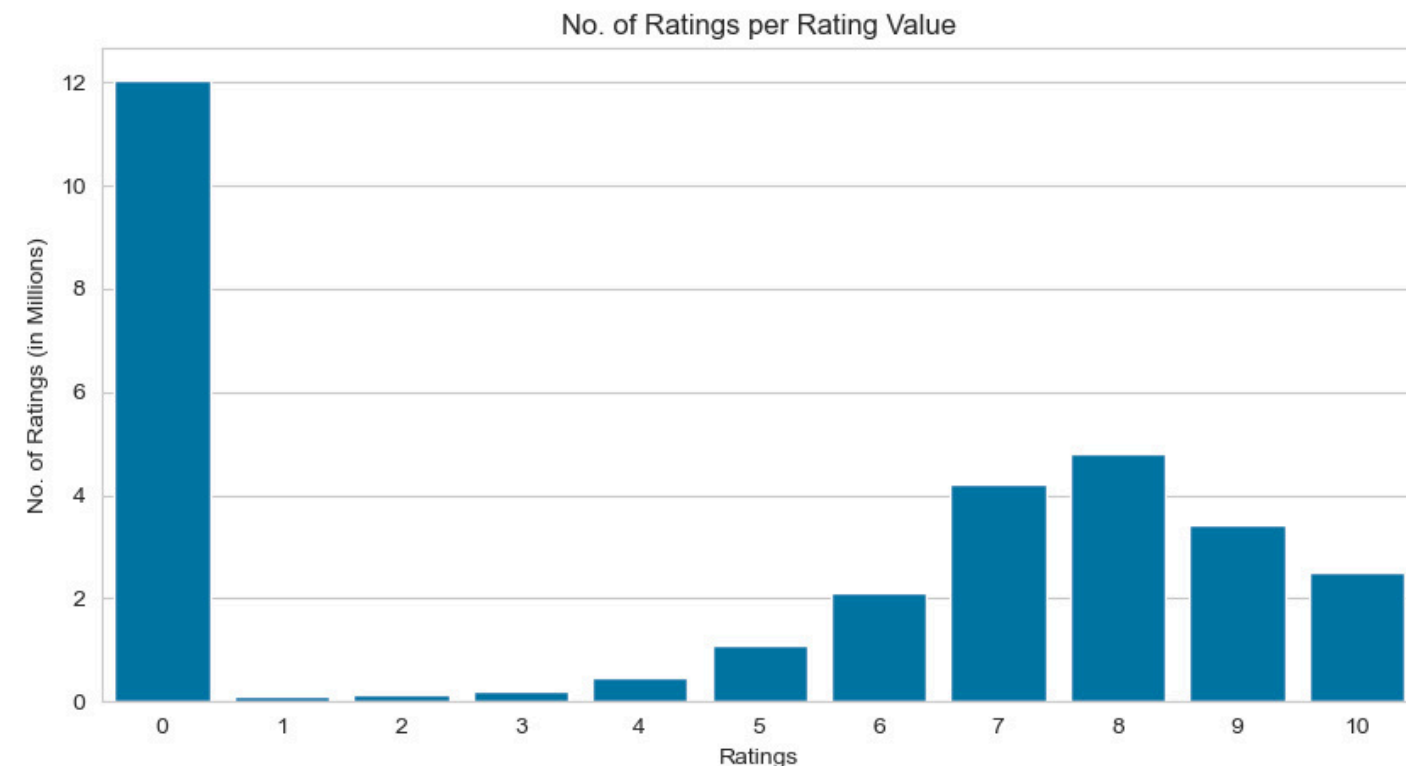


Before Data Cleaning

Dataset description



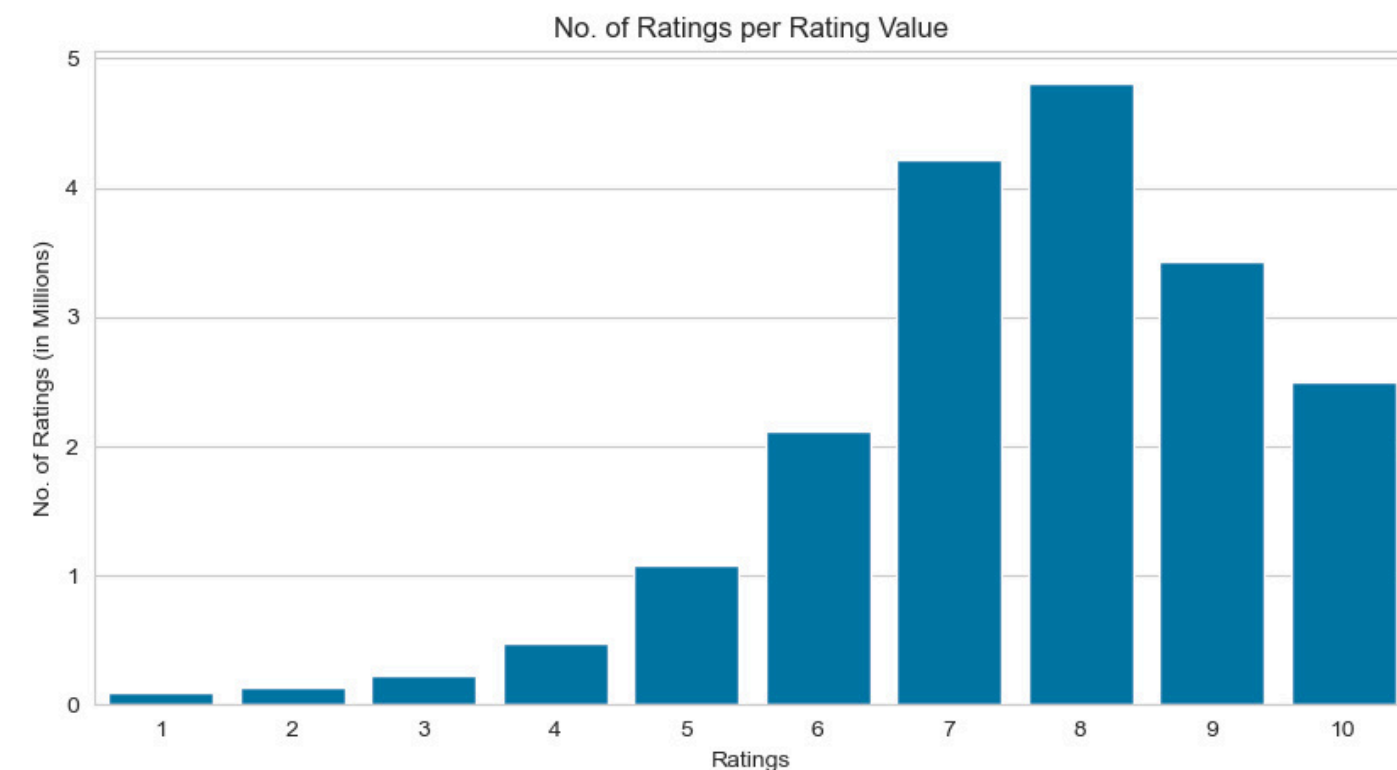
Cleaning rows w.r.t. my_score column in df_animelists data frame:



After Cleaning Rows with my_score column as 0.

Observation: The my_score column distribution shows that my_score with 0 value has many distributions.

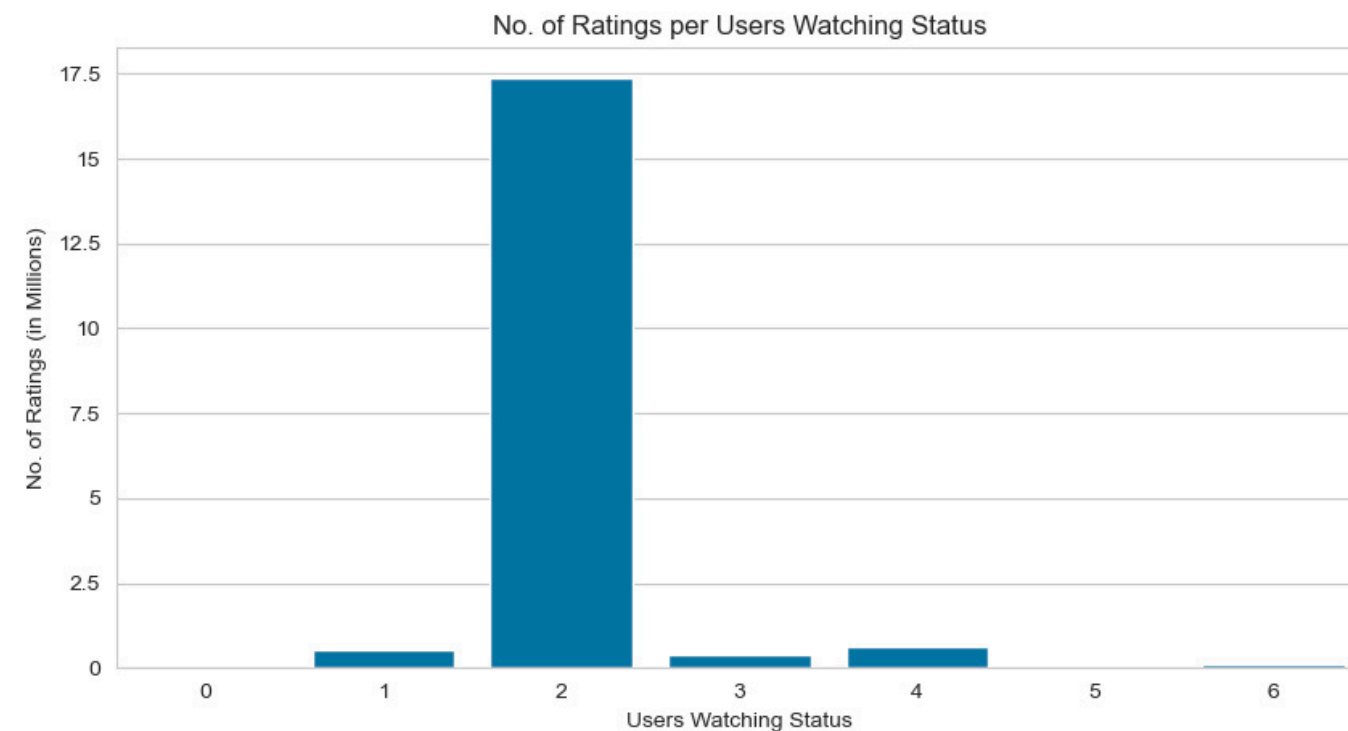
However, on the MyAnimeList website, users can only rate anime on a scale of 1 to 10. So, all the rows with a value of my_score equal to 0 have no significance, and we must remove such rows.



Dataset description



Cleaning rows with respect to my_status column in df_animelists:

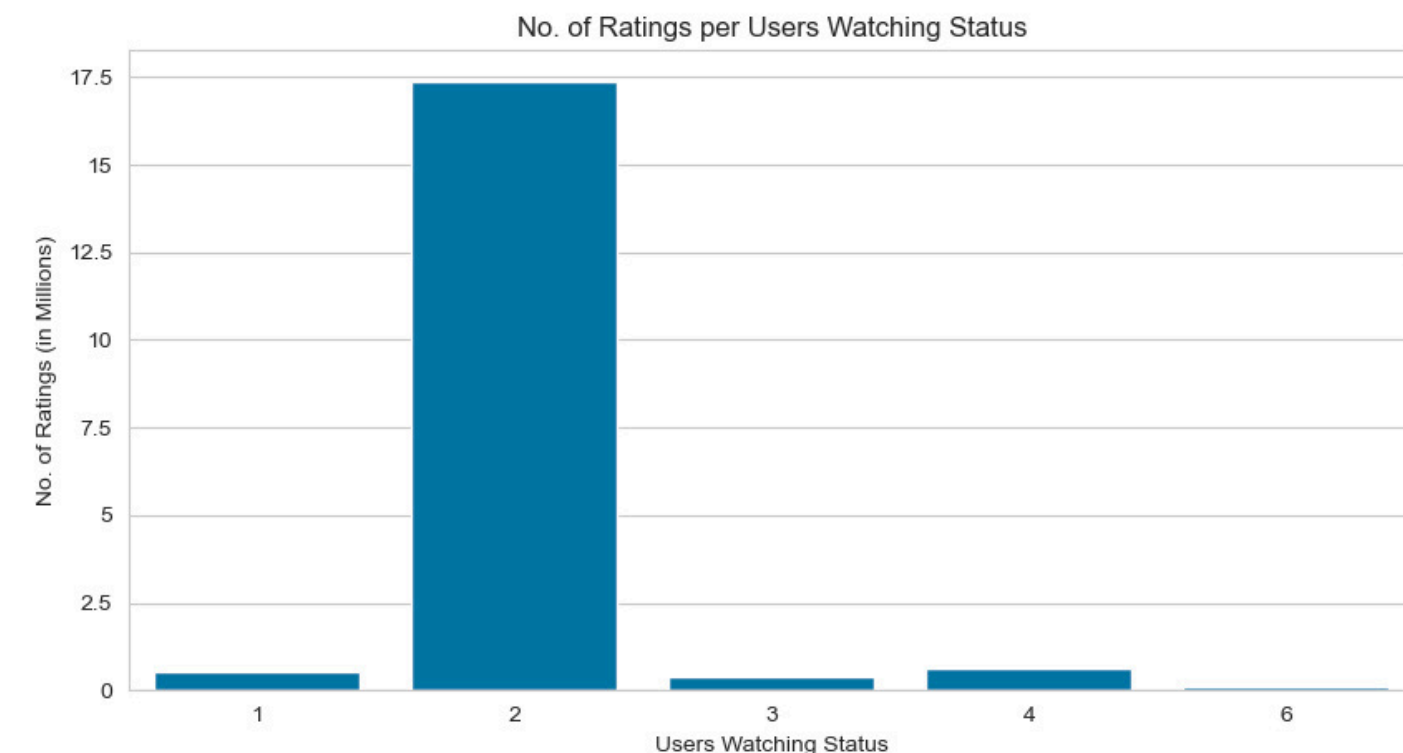


Observation: We removed rows having my_status value equal to 0 and 5, as it does not have any known significance to it.

my_status column in anime lists tables contains integer values, and their semantics are:

- 1: watching
- 2: completed
- 3: on hold
- 4: dropped
- 6: plan to watch

We will first check the my_status column, and if there are any other values apart from 1, 2, 3, 4, and 6, then we will remove such rows from the df_animelists data frame.



Building a Basic Similarity Matrix Model

A Similarity Matrix Model is a powerful tool used to measure the similarity between different data points or items based on some metric or criterion. In this model, a similarity matrix (often denoted as a square matrix) is constructed where each element represents the similarity between two data points. This concept is widely applied in domains like collaborative filtering for recommendation systems, clustering, text mining, etc.

Key Components of a Similarity Matrix Model:

- **Data Points/Items:** These are the entities between which similarity is measured. For example, in an anime recommender system, each data point could be an anime show, and the similarity matrix would measure how similar one anime is to another based on features such as genre, user ratings, etc.
- **Similarity Metric:** Common metrics used to populate the matrix include:
 - Cosine Similarity: Measures the cosine of the angle between two vectors. Useful for direction rather than magnitude.
 - Euclidean Distance: Measures the straight-line distance; lower values indicate higher similarity.
 - Pearson Correlation: Evaluates the linear relationship between two items.
 - Jaccard Similarity: Measures the intersection size over the union of sets for binary data.

Matrix Construction: A square matrix is created, where each element (i,j) shows the similarity between item i and item j . The diagonal elements are 1, indicating identical items.

Building a Basic Similarity Matrix Model

Example Workflow:

- **Preprocessing:**
 - Normalize or standardize the data (e.g., scaling feature vectors).
 - If dealing with textual data, you might use techniques like TF-IDF to vectorize the data.
- **Compute Similarity:**
 - Calculate pairwise similarity using a chosen metric (e.g., cosine similarity) to populate the matrix.
- **Post-Processing:**
 - Apply a threshold to filter the most relevant similarities (e.g., scores above 0.5).
- **Usage:**
 - **Clustering:** Group similar items based on high similarity scores.
 - **Recommendation:** Suggest items that are most similar to what users have interacted with.
 - **Visualization:** Use heatmaps to visualize item similarities.
- **Applications:** Anime Recommender Systems (Recommend shows based on features like genres and ratings.), Collaborative Filtering (Create user-item similarity matrices for personalized recommendations), Clustering (Use clustering algorithms on similarity matrices to group similar data points.)

Methodology



Building a Basic Similarity Matrix Model

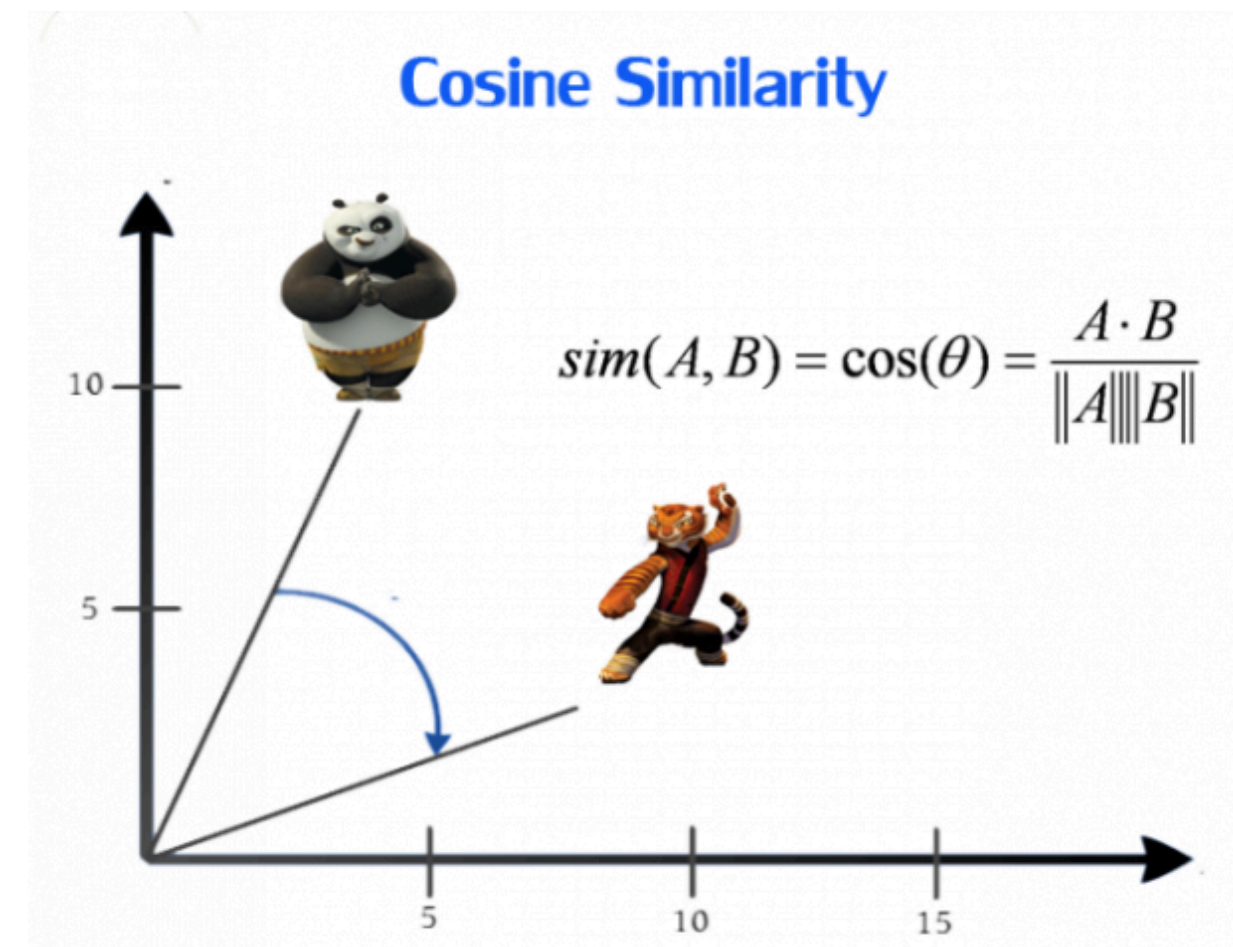
Creating Anime-Anime Similarity Matrix:

- The goal is to compute the similarity between different anime shows using cosine similarity, which measures how similar two non-zero vectors are regardless of their magnitude. The cosine similarity value ranges from -1 to 1, with 1 indicating identical vectors, 0 indicating no similarity, and -1 indicating opposing vectors.

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

where

- A and B are the vectors.
- A.B is the dot product.
- $\|A\|$ and $\|B\|$ are the magnitude of the vectors



Methodology



Content-Based Filtering

In Content-Based Filtering, we will use various anime features to create detailed profiles. The features include:

1. Anime Type (e.g., TV, Movie, OVA)
2. Anime Source (e.g., Manga, Light Novel, Original)
3. Anime Studio (e.g., Kyoto Animation, Madhouse)
4. Anime Genre (e.g., Action, Romance, Fantasy)

Step 1: Creating Anime Profiles

We will create **Anime Profiles** by applying **One-Hot Encoding** to these categorical features. This encoding helps convert each feature into a numerical vector, allowing us to represent each anime in a multi-dimensional feature space.

Step 2: Building User Profiles

To create User Profiles, we will sum up the anime profile vectors for all the anime a user has watched and multiply each anime vector by the rating given by the user to weigh their preferences.

This process ensures that the user's profile captures their preferences based on the types of anime they watch and their ratings.

Step 3: Finding Recommendations

Finally, we will apply **Cosine Similarity** between the User Profile and Anime Profiles to identify recommendations. This similarity score helps us find the anime that best aligns with each user's preferences, ensuring personalized recommendations.

Methodology



Collaborative Filtering

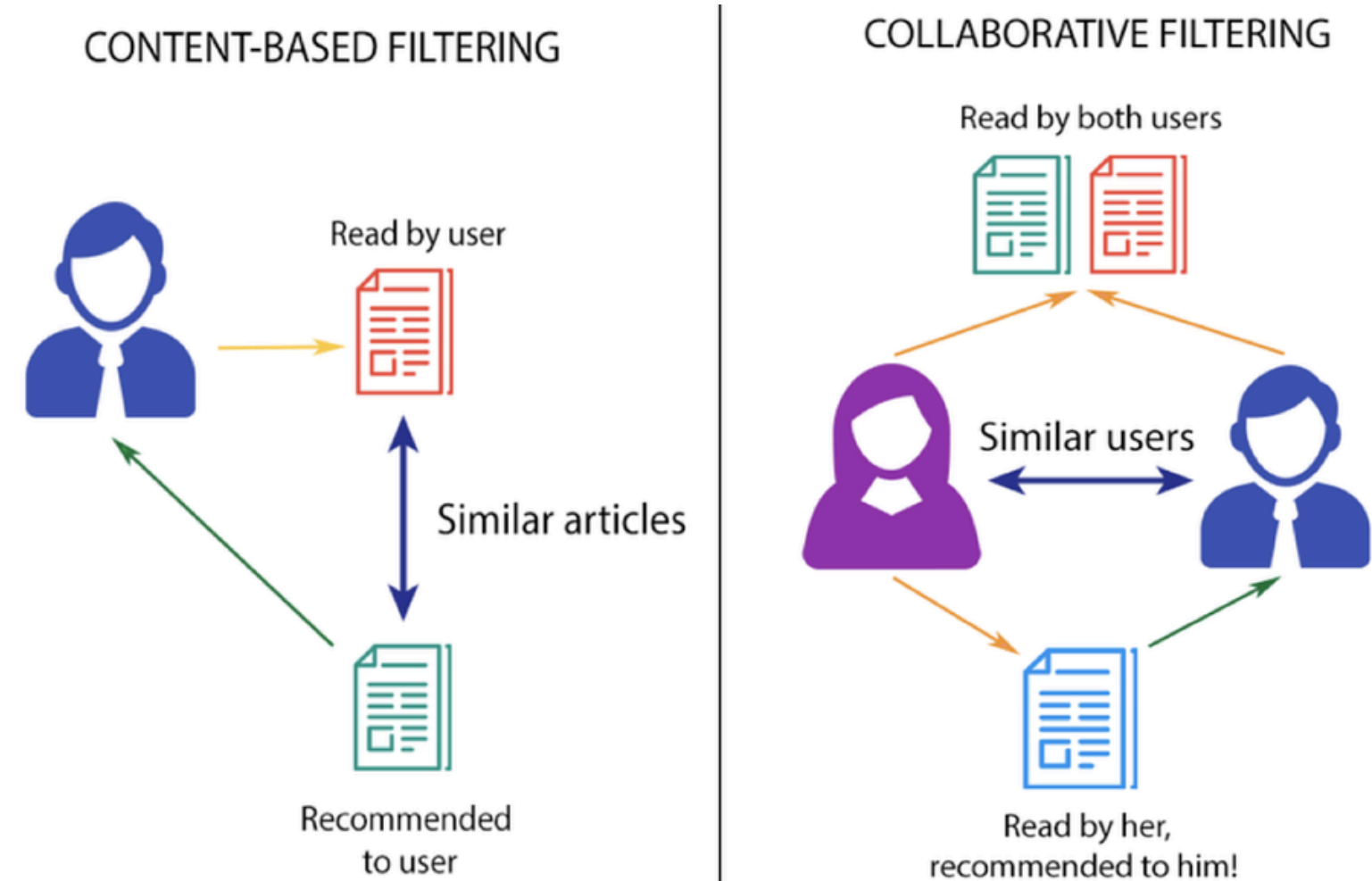
Collaborative filtering is a technique for recommendation systems that suggests items based on user preferences or behaviors. There are two main types:

1. **User-Based Collaborative Filtering:** Recommends items to a user based on similar users' preferences.
2. **Item-Based Collaborative Filtering:** Recommends items the user has already liked.

Key Steps:

- Calculate similarity (e.g., cosine similarity or Pearson correlation).
- Find similar users/items.
- Predict ratings for unseen items.
- Recommend items with high predicted ratings.

Challenges include the cold start problem, data sparsity, and scalability. Hybrid approaches improve accuracy



Collaborative Filtering

KNNBasic Using Surprise Library (Anime-Anime Similarity)

KNNBasic is used in item-based collaborative filtering to predict user ratings for anime based on similar anime the user has already rated. The method finds K similar anime (neighbors) for an anime and predicts ratings using weighted averages based on similarity.

Steps:

1. Find K similar anime: Identify K anime similar to an unrated anime i , based on user ratings, using the Pearson correlation coefficient:

$$\text{sim}(i, j) = \frac{\sum_{v \in U} (r_{v,i} - \bar{r}_i)(r_{v,j} - \bar{r}_j)}{\sqrt{\sum_{v \in U} (r_{v,i} - \bar{r}_i)^2} \sqrt{\sum_{v \in U} (r_{v,j} - \bar{r}_j)^2}}$$

where $r_{v,i}$ is the rating user v gave to anime i , and $\bar{r}_i$ is the average rating for anime i .

Collaborative Filtering

Steps:

2. Predict the rating $\hat{r}_{u,i}$ for user u by taking the weighted average of user u's ratings for the K similar anime j:

$$\hat{r}_{u,i} = \frac{\sum_{j \in K} \text{sim}(i, j) \cdot r_{u,j}}{\sum_{j \in K} |\text{sim}(i, j)|}$$

Here, $r_{u,j}$ is the rating user u gave to anime j, and the weight is based on the similarity $\text{sim}(i,j)$.

Pearson Correlation Coefficient:

The Pearson correlation coefficient is commonly used in collaborative filtering because it captures how two anime are similarly rated by multiple users. It normalizes the ratings by subtracting the mean, ensuring that differences in user rating scales are taken into account. This allows the algorithm to focus on the patterns in user preferences rather than absolute rating values.

Collaborative Filtering

KNNBaseline using Surprise Library

In the context of collaborative filtering, the KNNBaseline algorithm enhances the basic K-Nearest Neighbors (KNN) approach. It incorporates a **baseline rating** into the prediction to account for the inherent biases in user and item ratings. This baseline adjusts for factors like a user's general tendency to rate items higher or lower than average and certain anime being rated more highly across the board.

Steps:

1. **Baseline Estimate:** Compute a baseline rating as

$$b_{u,i} = \mu + b_u + b_i$$

where μ is the overall average rating.

b_u is the user bias (how much user u tends to rate higher/lower than average).

b_i is the item bias (how anime i is rated compared to average).

Collaborative Filtering

KNNwithMeans using Surprise Library

In collaborative filtering, the KNNWithMeans algorithm builds upon basic KNN by incorporating the mean rating of each user into the prediction process. This approach helps in adjusting for individual differences in rating scales, as some users tend to give higher or lower ratings on average. By considering how much a user's rating deviates from their personal average, the algorithm aims to generate more accurate recommendations.

Steps:

1. **Calculate the mean rating:** The first mean rating $\bar{r}_u$ of user u across all the anime they have rated is computed as:

$$\bar{r}_u = \frac{\sum_{i \in I_u} r_{u,i}}{|I_u|}$$

where I_u is the set of anime rated by user u , and $r_{u,i}$ is the rating that user u gave to anime i .

Collaborative Filtering

Singular Value Decomposition (SVD) using Surprise Library

In the Surprise Library, Singular Value Decomposition (SVD) is a matrix factorization technique used in collaborative filtering. The SVD model predicts the rating a user u would give to an anime i by factoring both users and items (anime) into lower-dimensional latent factor spaces. The model combines the global average rating, user and item biases, and latent factors to generate the prediction.

Predicted Rating in the SVD Model: The predicted rating $\hat{r}_{u,i}$ for user u on anime i in the SVD model is given by:

$$\hat{r}_{u,i} = \mu + b_u + b_i + q_i^T p_u$$

where:

- μ is the global average rating across all users and items in the training data.
- b_u is the user bias, indicating how much user u rates items compared to the average.
- b_i is the item bias (anime bias), showing how anime i is generally rated compared to the average.
- p_u is the user vector representing user u 's preferences in a latent factor space.
- q_i is the item vector representing anime i in the same latent factor space.

Collaborative Filtering

Machine Learning Model Training Using Singular Value Decomposition (SVD)

The project employs Singular Value Decomposition (SVD) to extract latent features for each user and anime based on their interactions. These features serve as vectorized representations for building machine learning models in a Model-Based Collaborative Filtering approach. The following models are explored:

1. Linear Regression
2. Support Vector Regression (SVR)
3. Gradient Boost Decision Tree (GBDT) using LightGBM

Data Preparation:

1. Data Matrix: The input data is organized as a matrix where:
 - a. Rows represent users.
 - b. Columns represent anime IDs.
 - c. Matrix entries contain user ratings for each anime (0 if unrated).
2. Matrix Factorization with SVD:
 - a. The data matrix is decomposed into three components:
 - U : User feature matrix.
 - Σ : Diagonal matrix of singular values.
 - V^T : Anime feature matrix.
 - b. These components are used to reconstruct a low-rank approximation of the original matrix, capturing the latent structure.
3. Feature Extraction: Each user ID and anime ID is mapped to a vector extracted from the SVD components.

Collaborative Filtering

Machine Learning Model Training Using Singular Value Decomposition (SVD)

Train-Test Split:

1. The dataset is divided into X_{train} and X_{test} , where:
 - a. X : Vectorized representations of users and animes.
 - b. y : Corresponding ratings from the original data matrix.
2. Random splitting or time-based splitting is used depending on the application context.

Machine Learning Models:

1. Linear Regression:
 - a. Used to establish a baseline by modeling the relationship between user-anime vectors and their ratings.
 - b. Simple and interpretable model suitable for initial experimentation.
2. Support Vector Regression (SVR):
 - a. Utilizes a kernel-based approach to handle non-linear relationships in the data.
 - b. Effective in capturing complex user preferences.
3. Gradient Boost Decision Tree (GBDT) using LightGBM:
 - a. Employs gradient boosting to iteratively minimize prediction error.
 - b. Handles large-scale datasets efficiently and captures intricate patterns in user preferences.

Methodology



Hybrid Recommendation System

1. Content-Based Filtering:
 - Vectorizes features (type, source, studio, genre) and computes cosine similarity with user profiles.
 - Ratings are scaled to 1–10.
2. Collaborative Filtering (KNNBaseline):
 - Uses precomputed predictions for existing users.
3. Hybrid Ratings:
 - Combines 3% content-based and 97% KNNBaseline ratings for existing users.
 - For new users, uses only content-based ratings.
4. Recommendations:
 - Filters out watched anime.
 - Sorts and selects top N anime based on hybrid ratings.

Key Features:

- Combines strengths of content-based and collaborative filtering.
- Handles both new and existing users effectively.



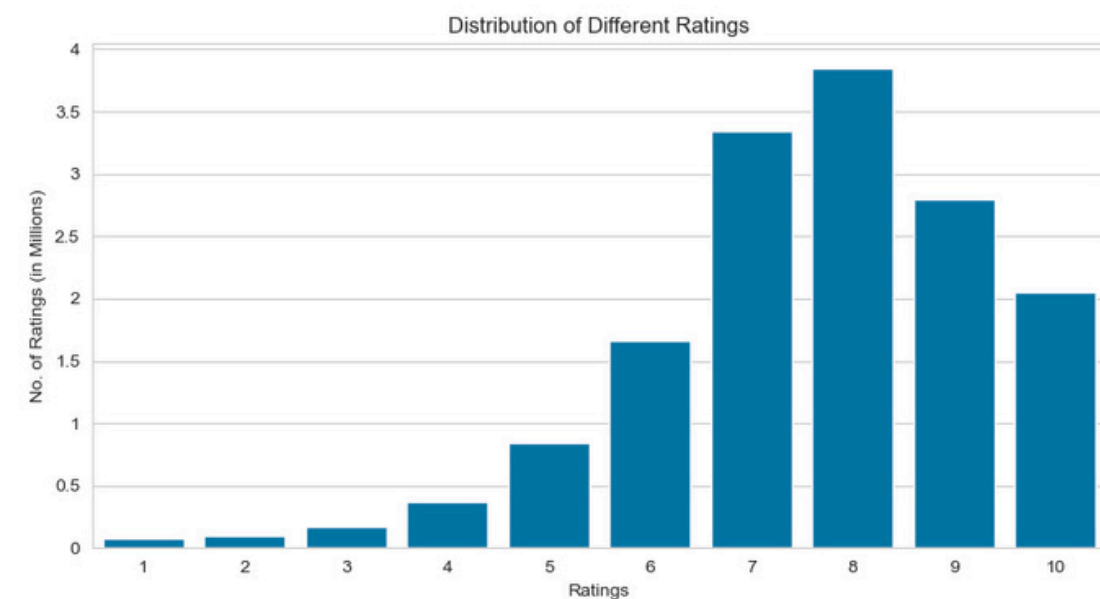
Results/Analysis



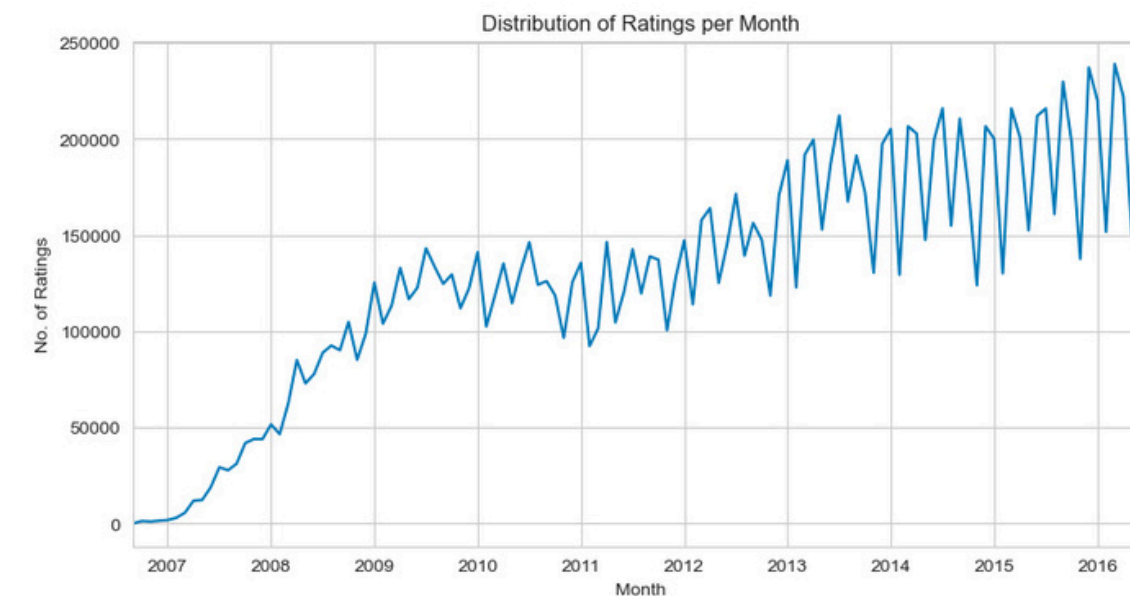
Mean value of all the rating given by the user in Training Dataset : 7.625716028258655

Mean value of all the rating given by the user in Test Dataset : 7.458522332112329

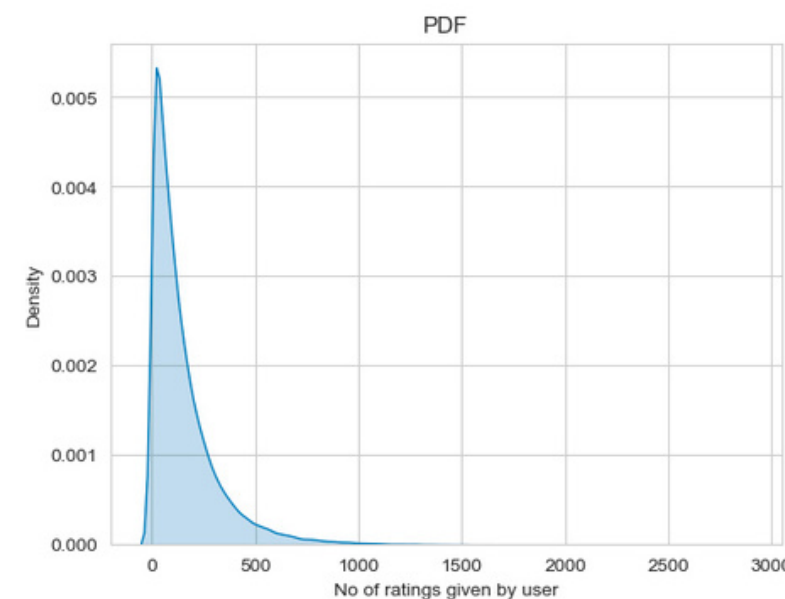
Distribution of Ratings given by Users:



Distribution of Ratings given by Users:

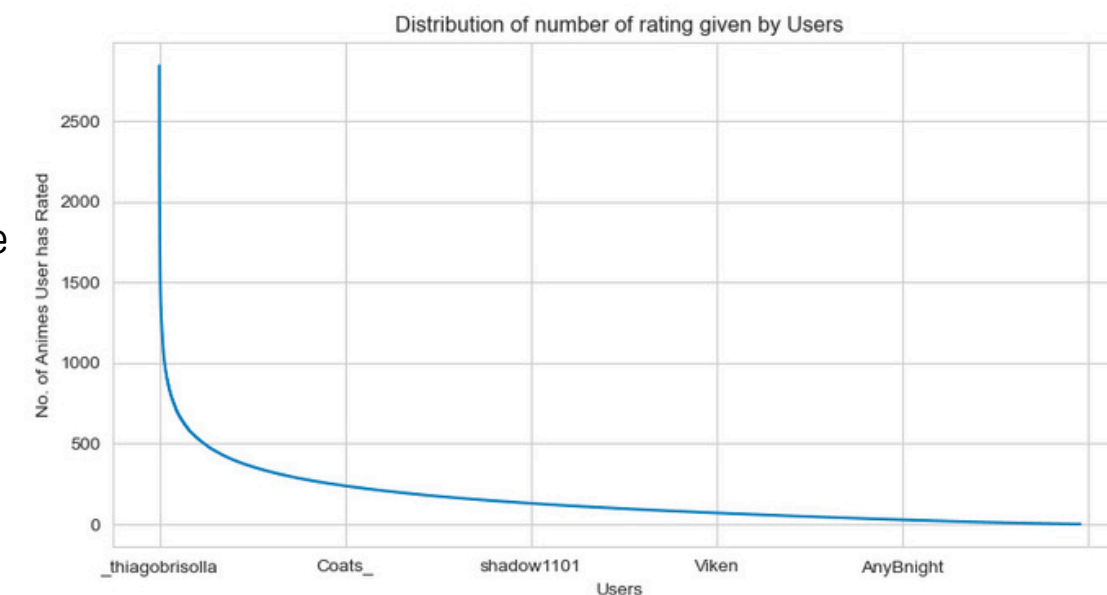


Analysis of Ratings given by Users:



Observation

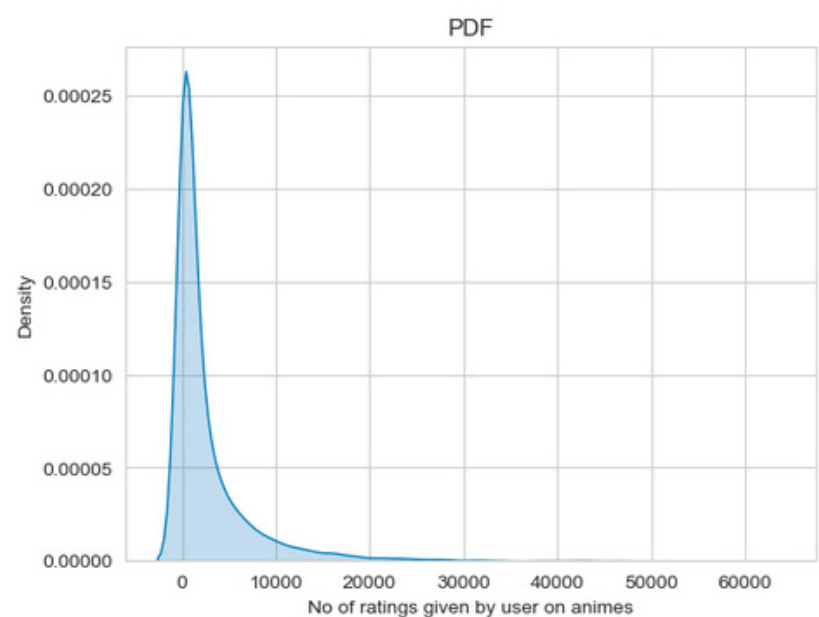
- The most number of anime rated by a single user in training dataset is 2845 anime.
- The least number of anime rated by a single user in training dataset is 1.



Results/Analysis

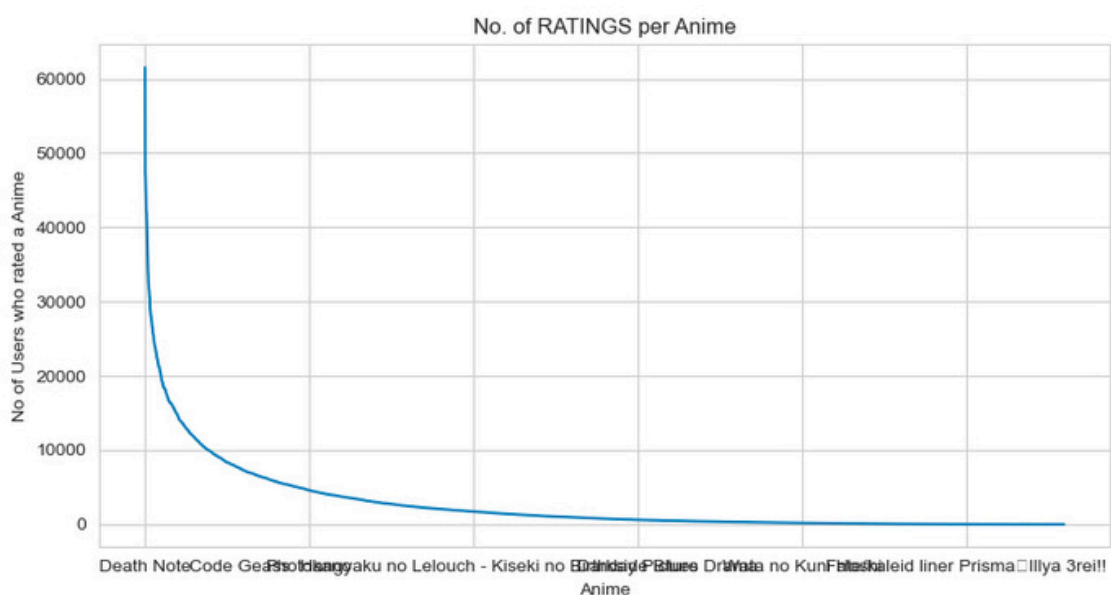


Analysis of Animes as Rating given by Users:

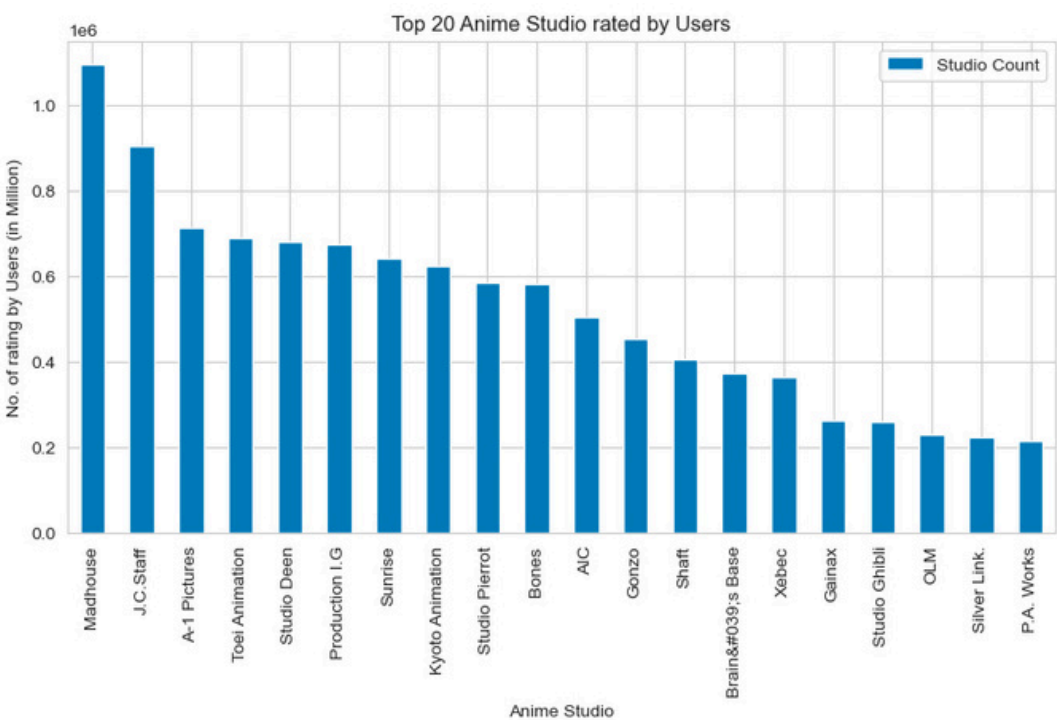
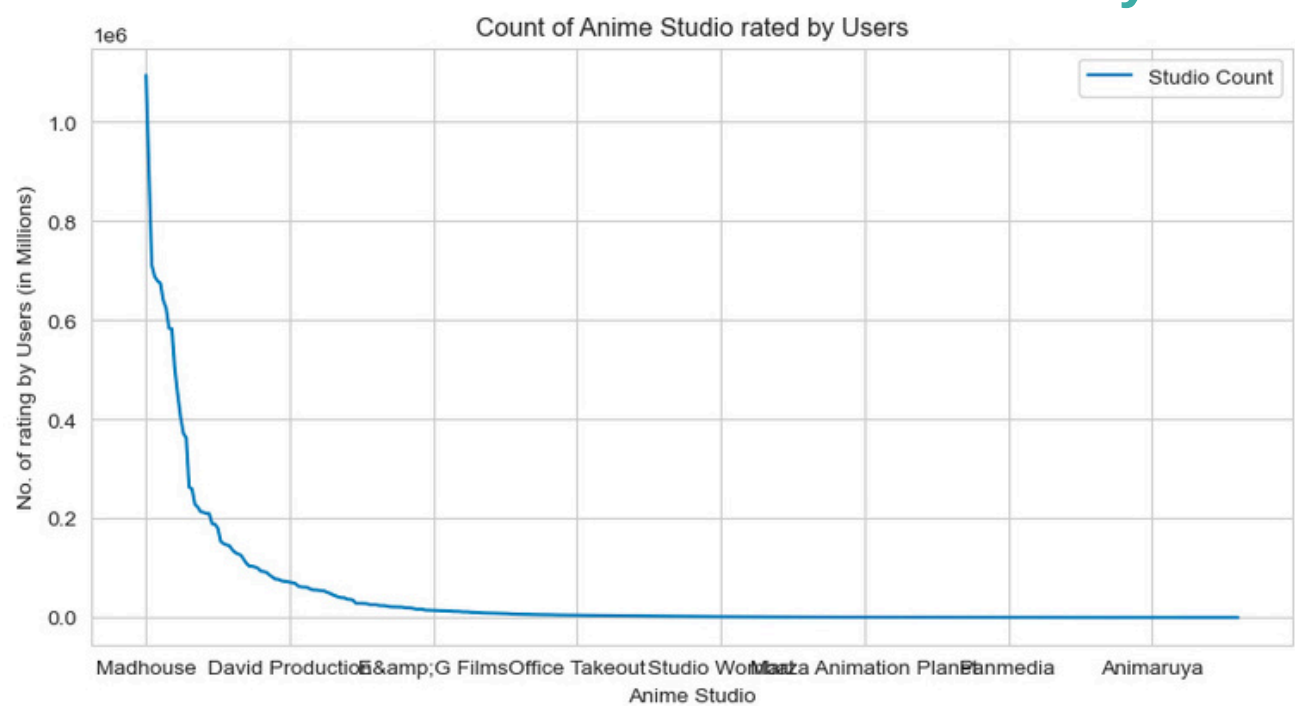


Observation

- Death Note is the most popular and rated Anime by Users.



Analysis of Rating on Anime Studios:

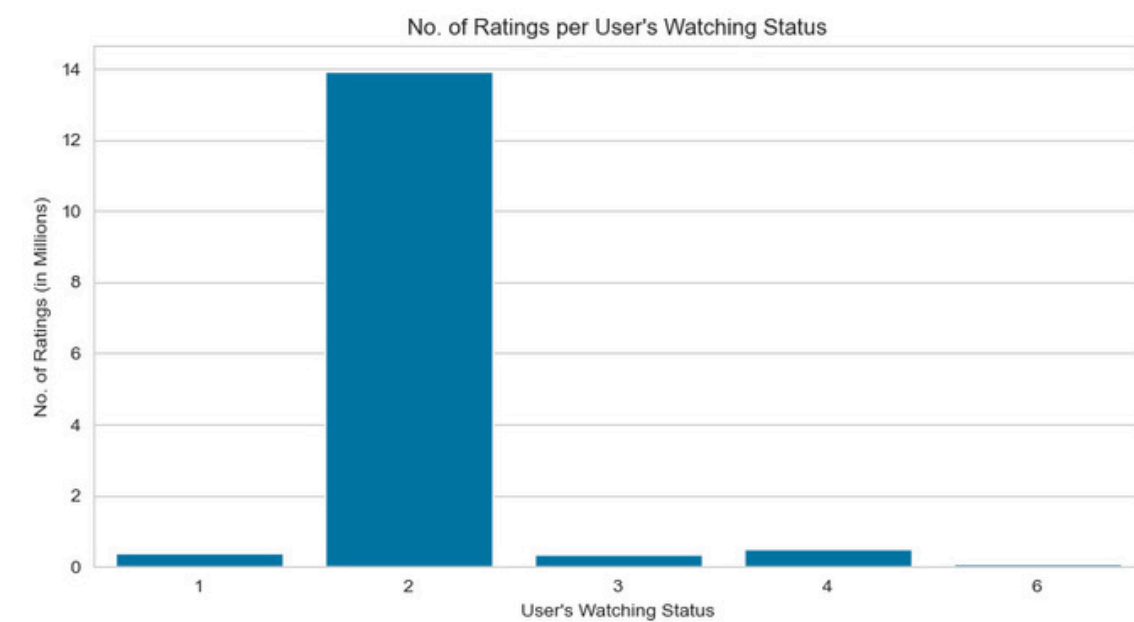
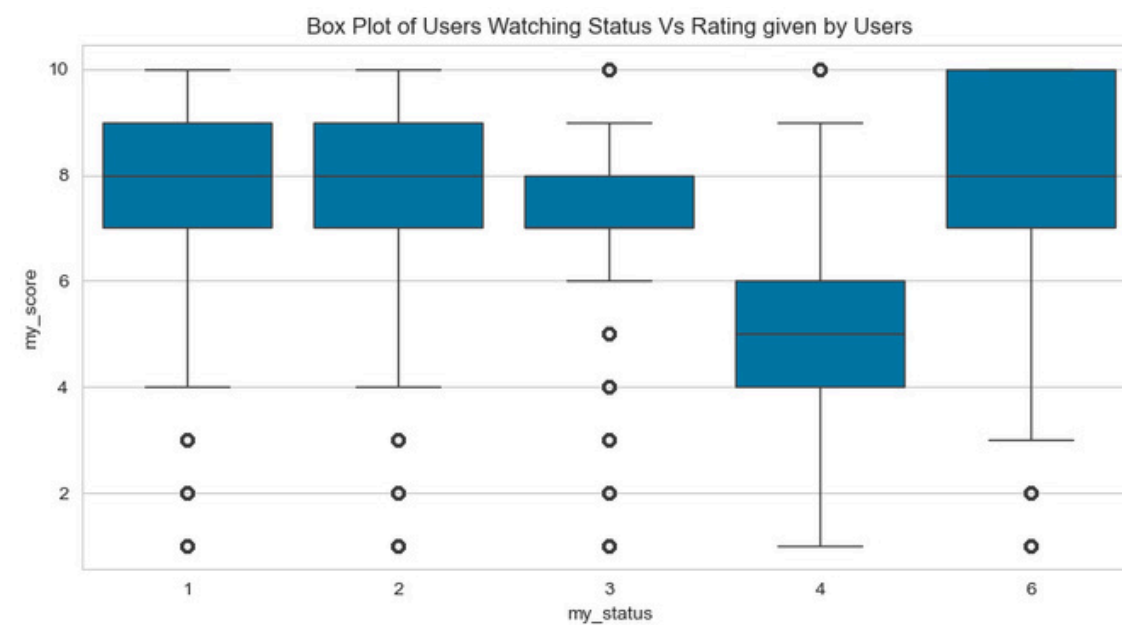
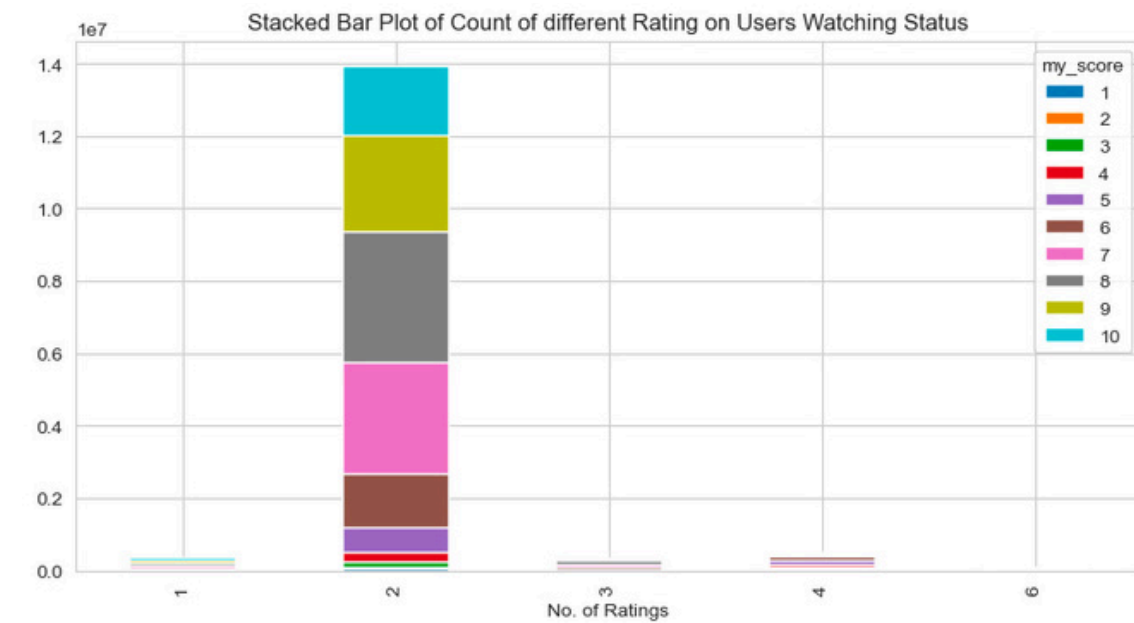
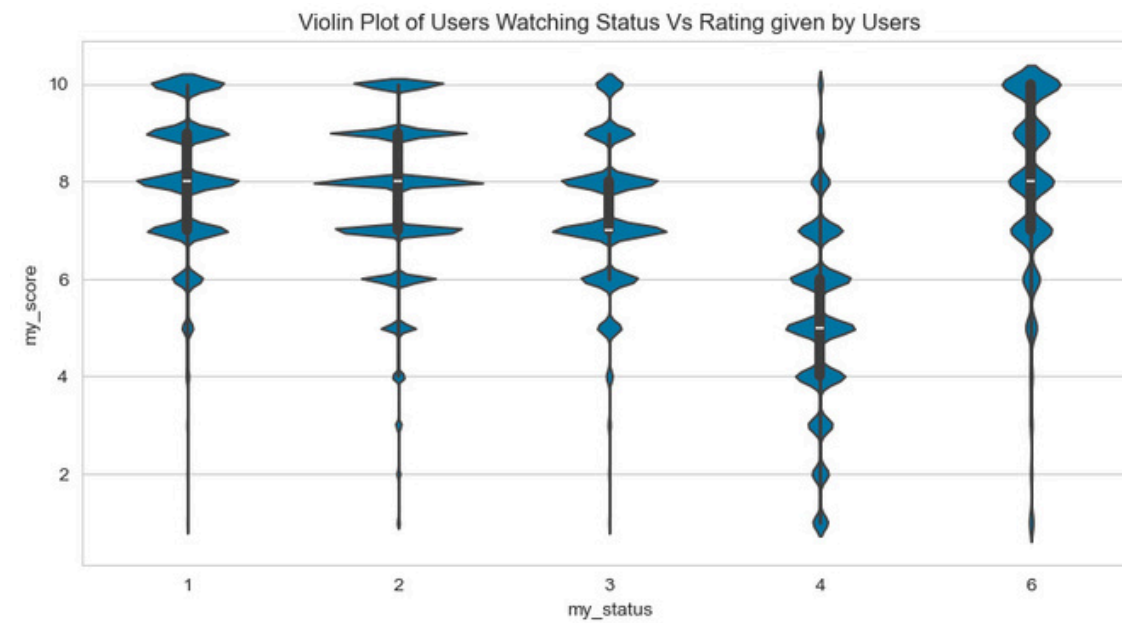


Results/Analysis



Analysis of Rating on Users Watching Status:

The average rating for my_status = 4 is relatively lower than that of other statuses, while the most frequently rated value by users for my_status = 2 is higher

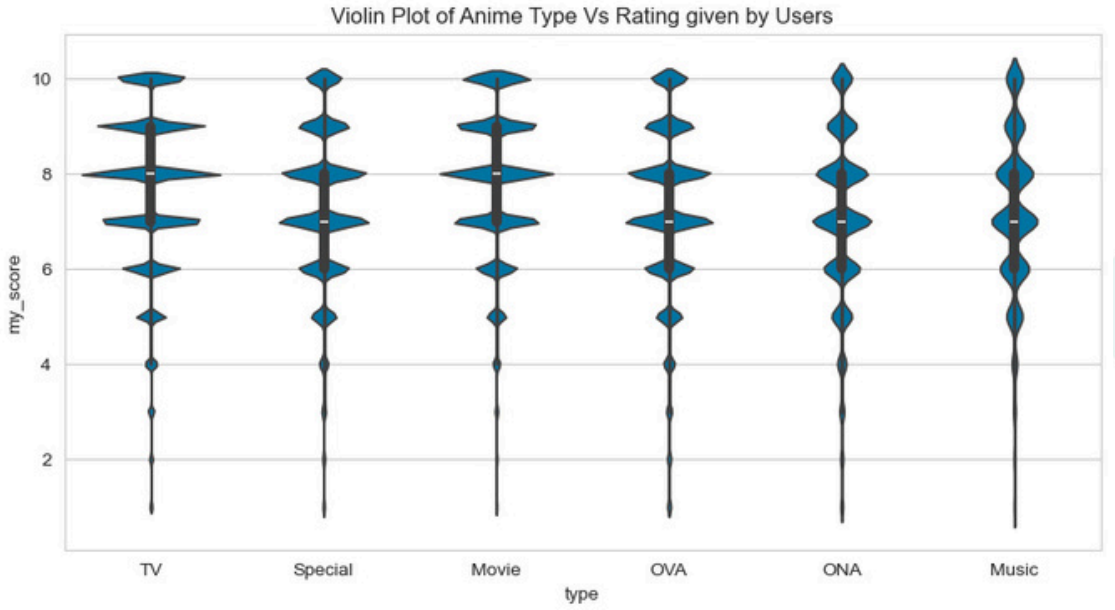
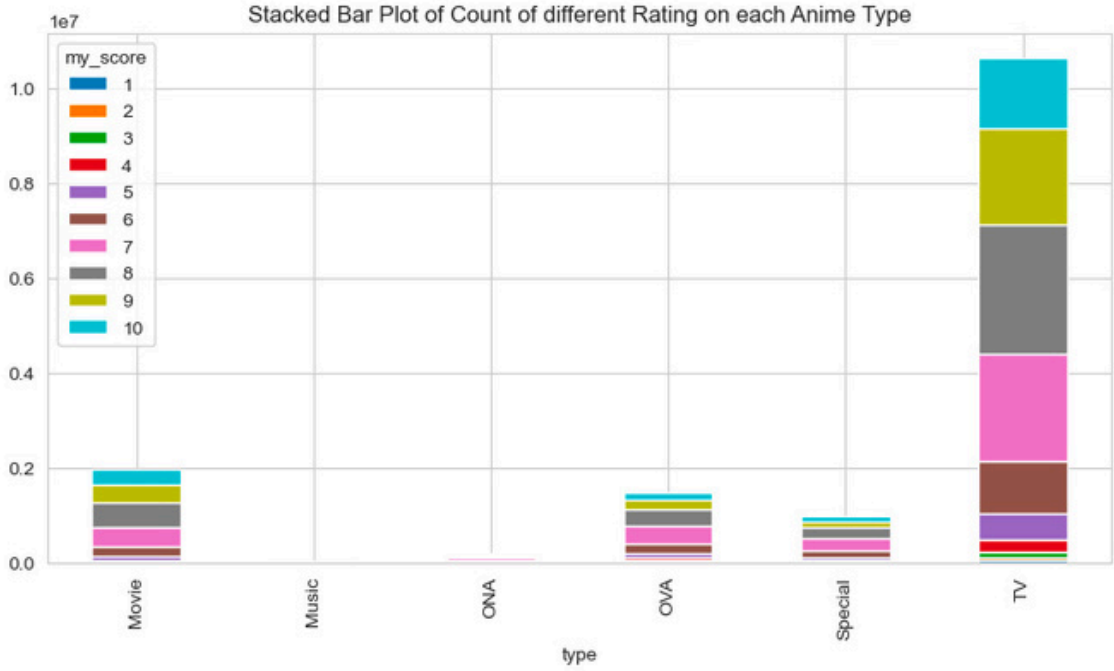
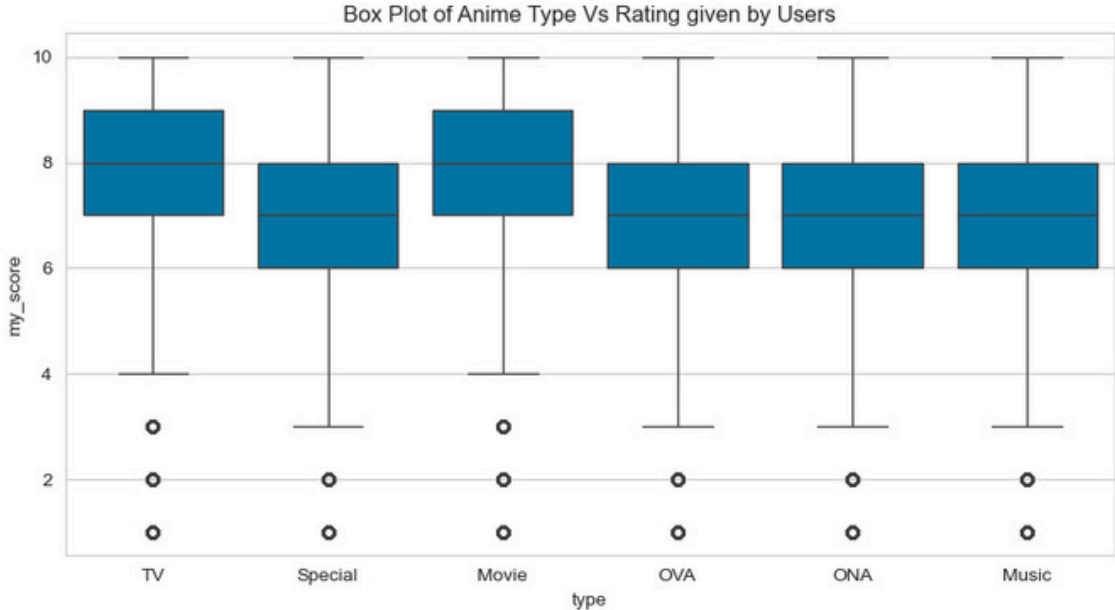
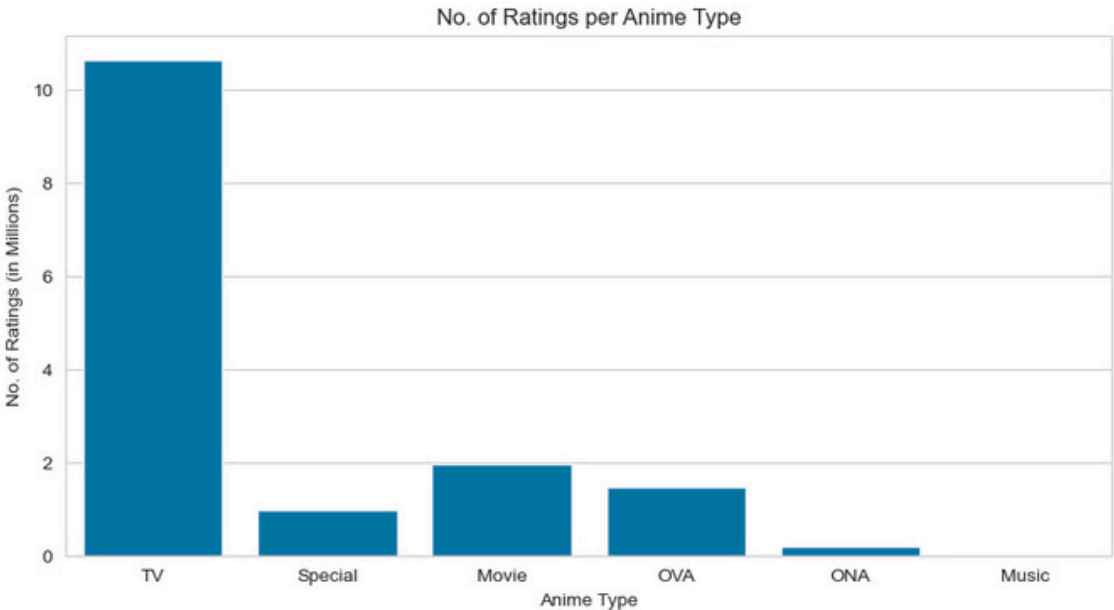


Results/Analysis



Analysis of Rating on Anime Type:

It can be seen that in each anime type has different most rated value i.e. 7 or 8.

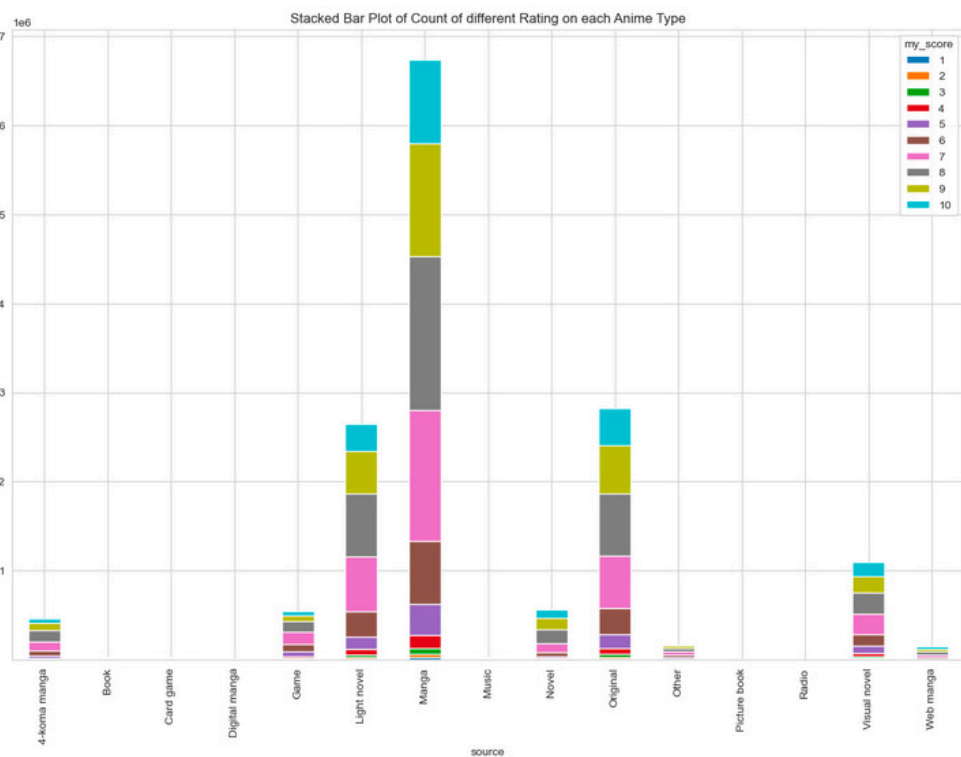
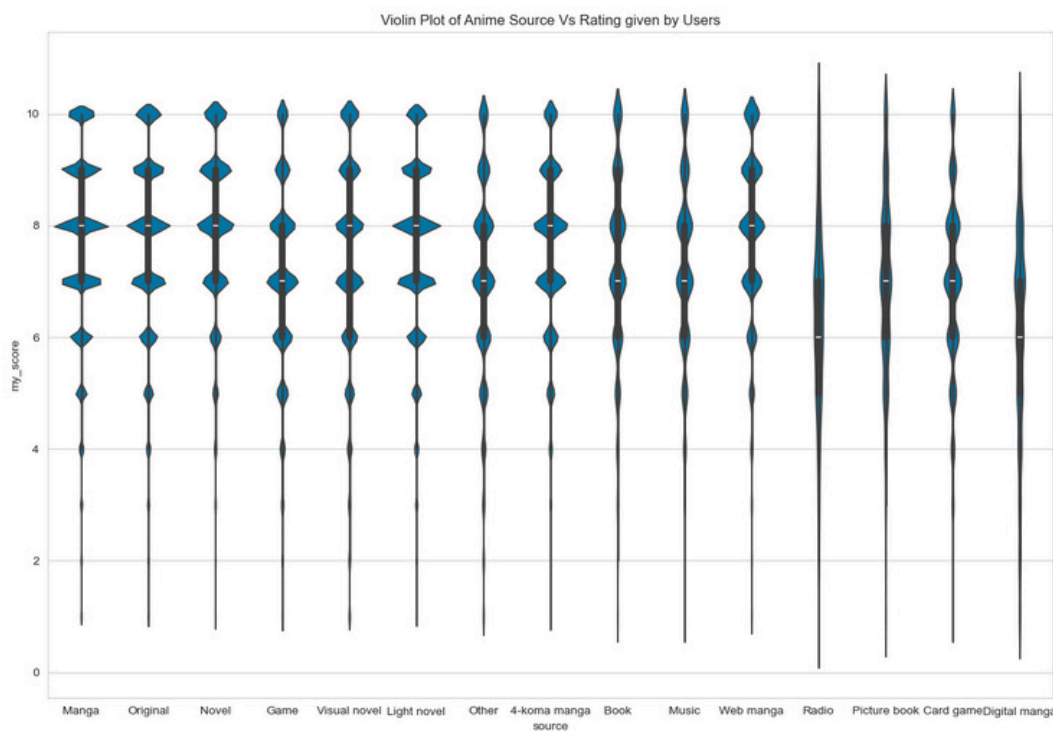
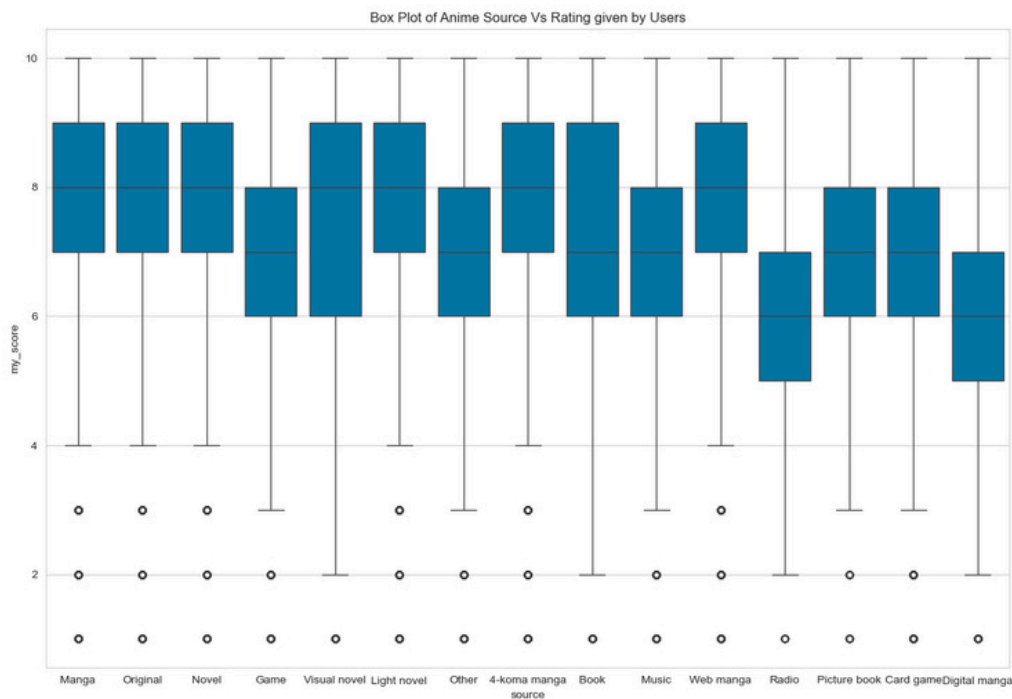
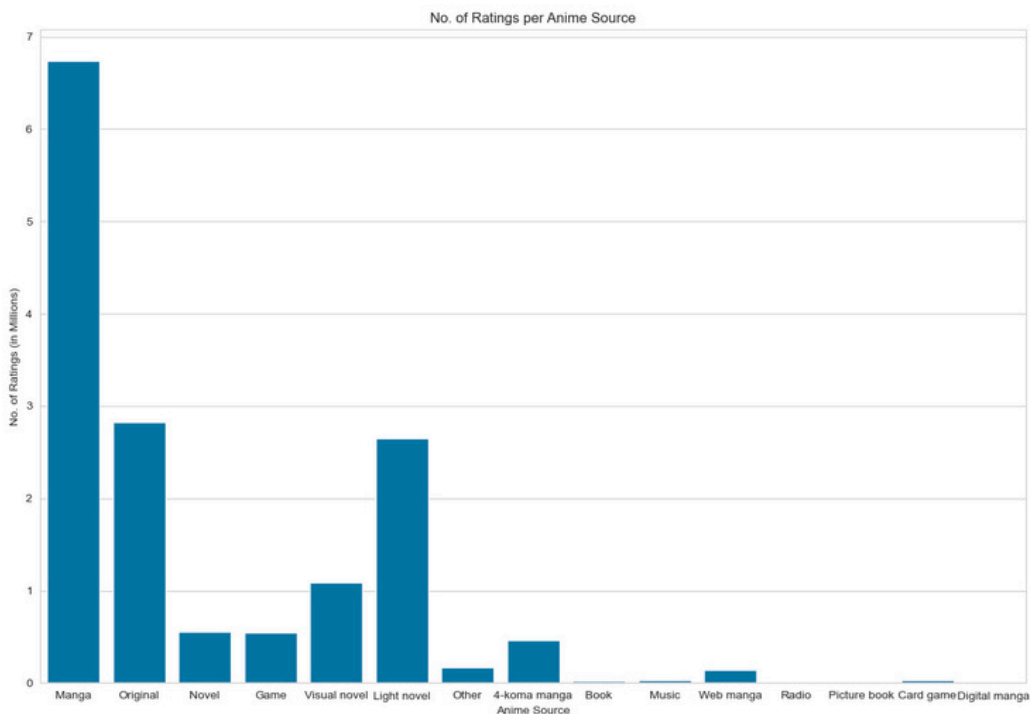


Results/Analysis

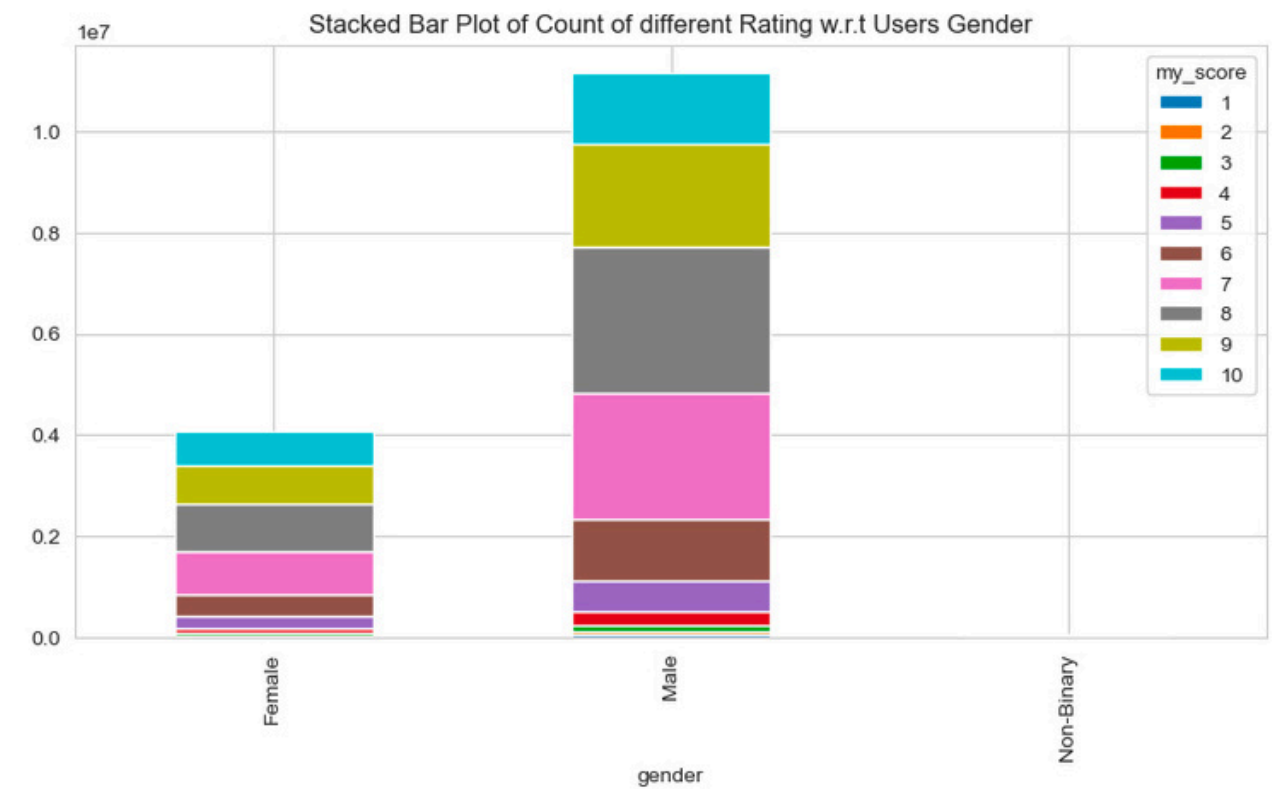
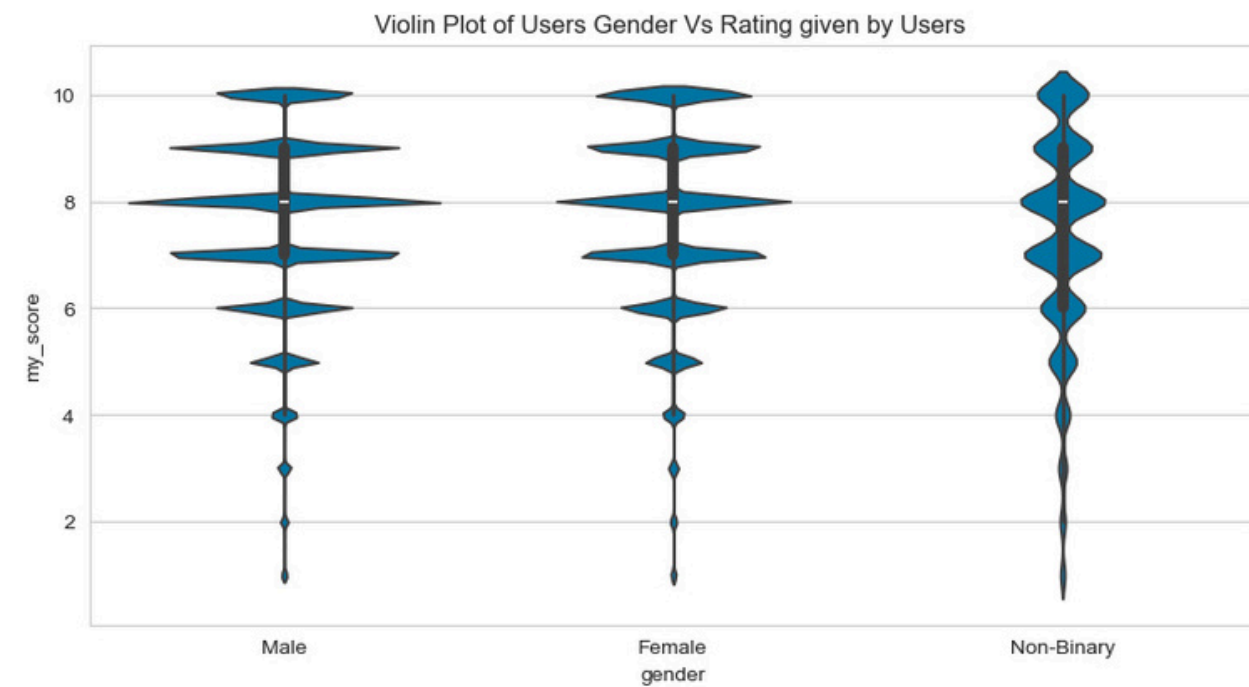
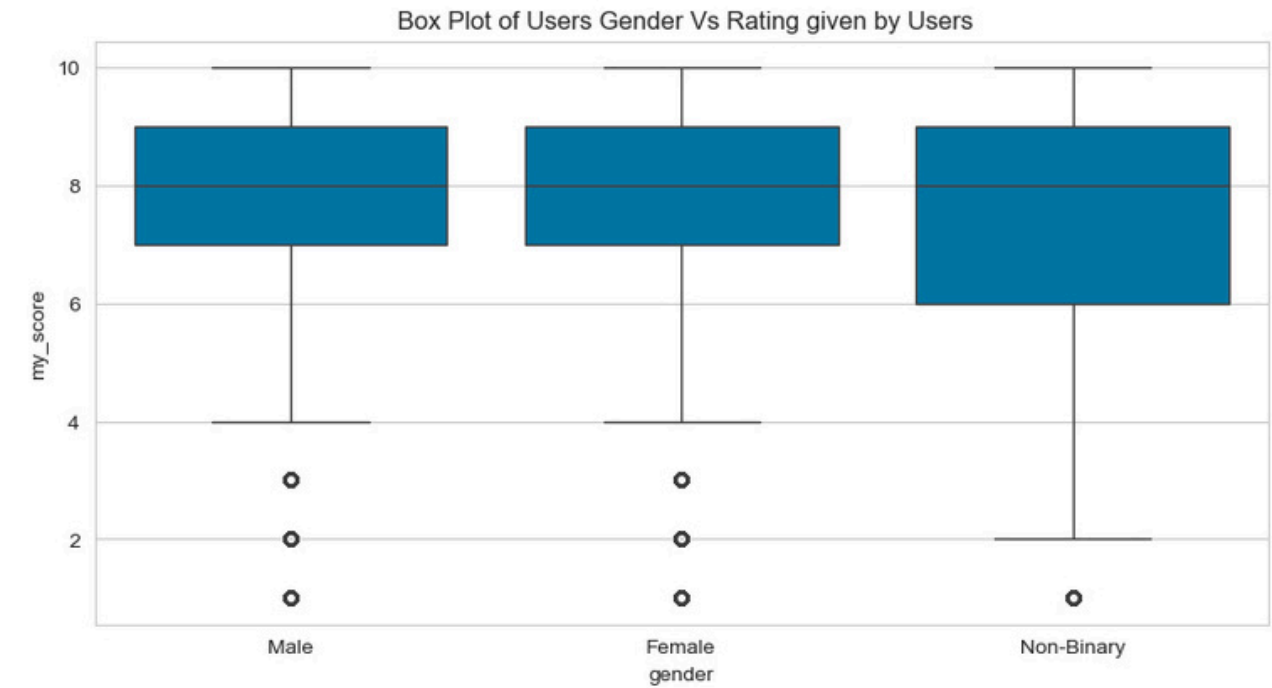
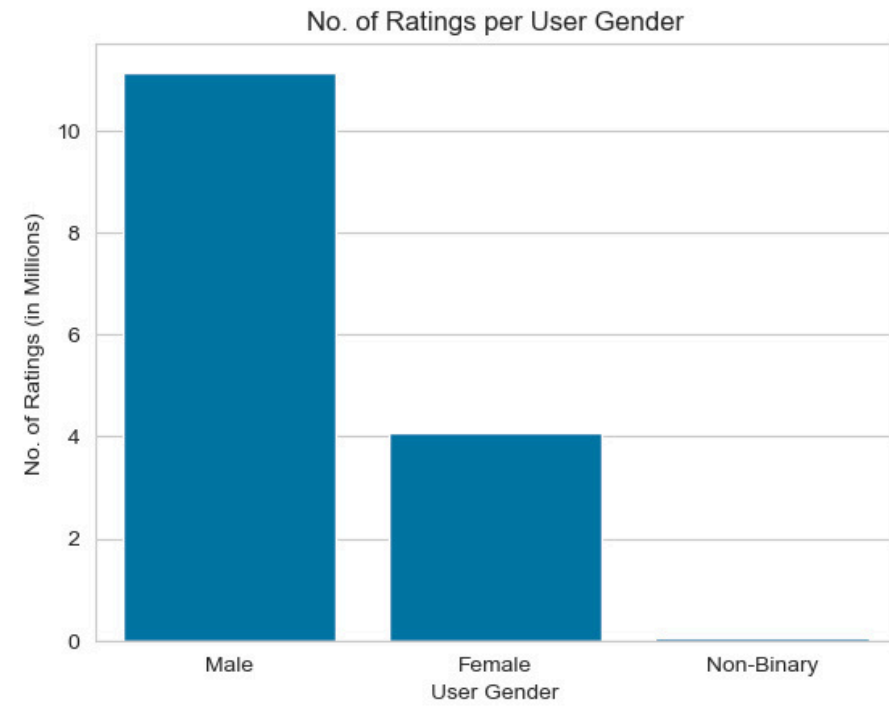


Analysis of Rating on Anime Source:

It can be seen that each anime sources has different most rated value i.e. 7 or 8.



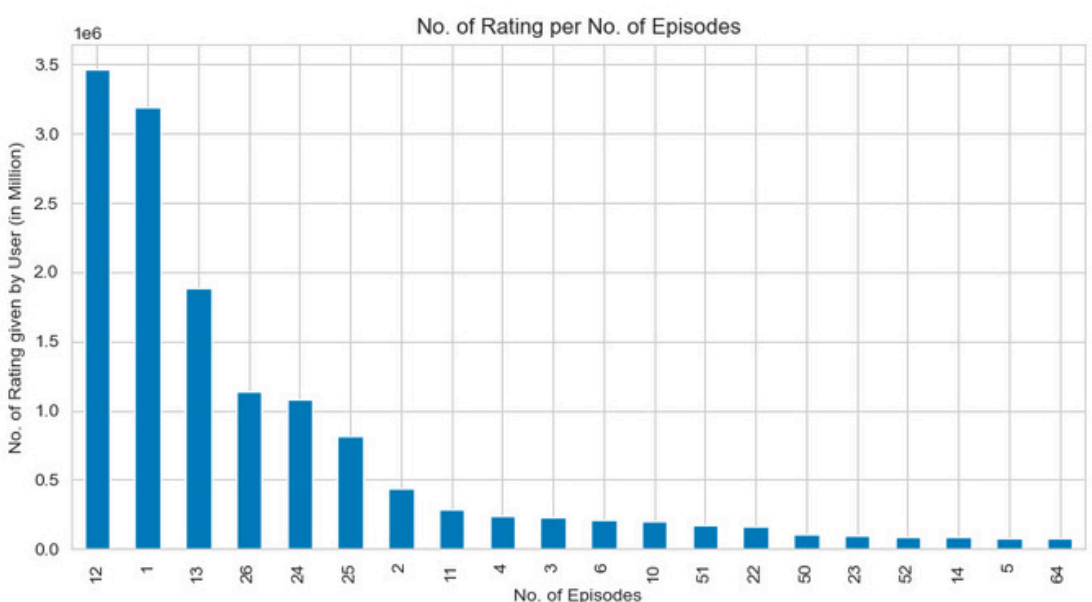
Analysis of Rating on Users Gender: Various animes with more than one studio.



Results/Analysis

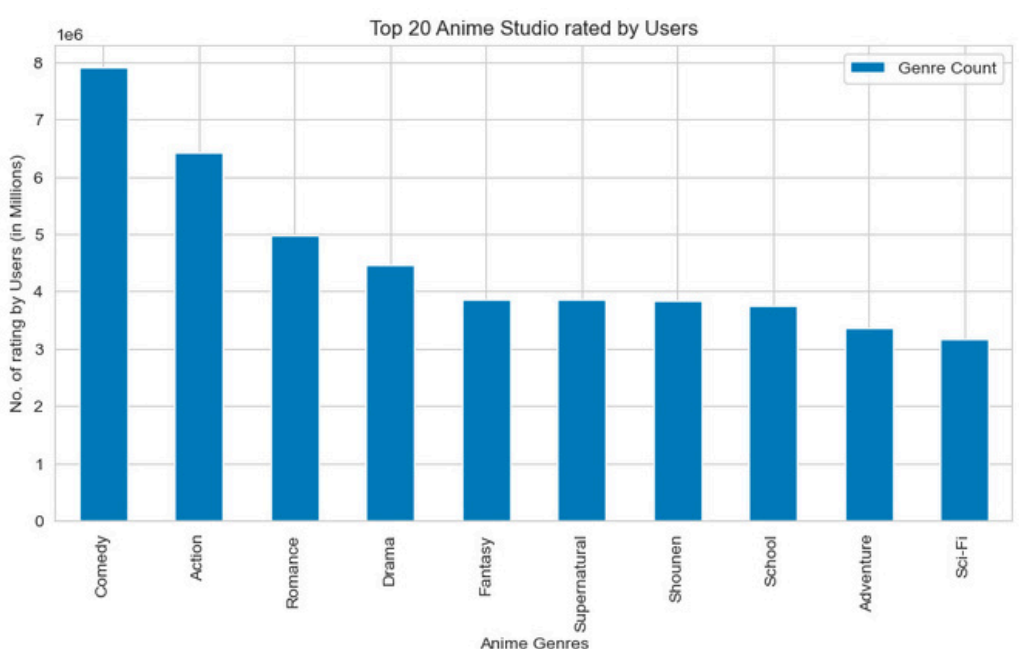


Analysis of Rating on Total No. of Episodes in each Anime:

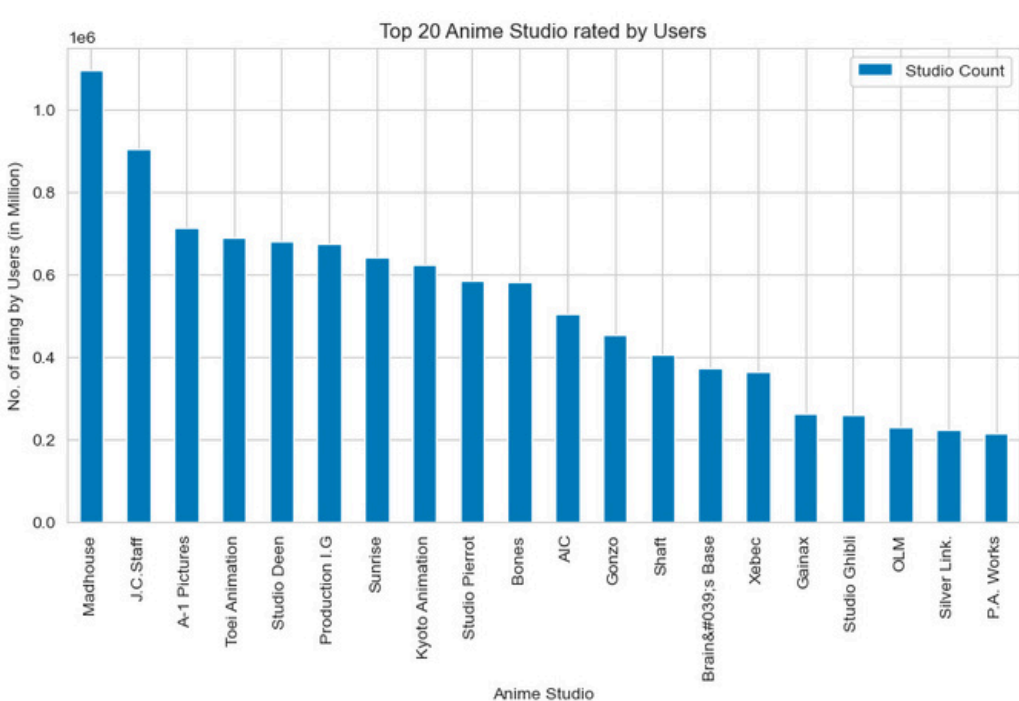
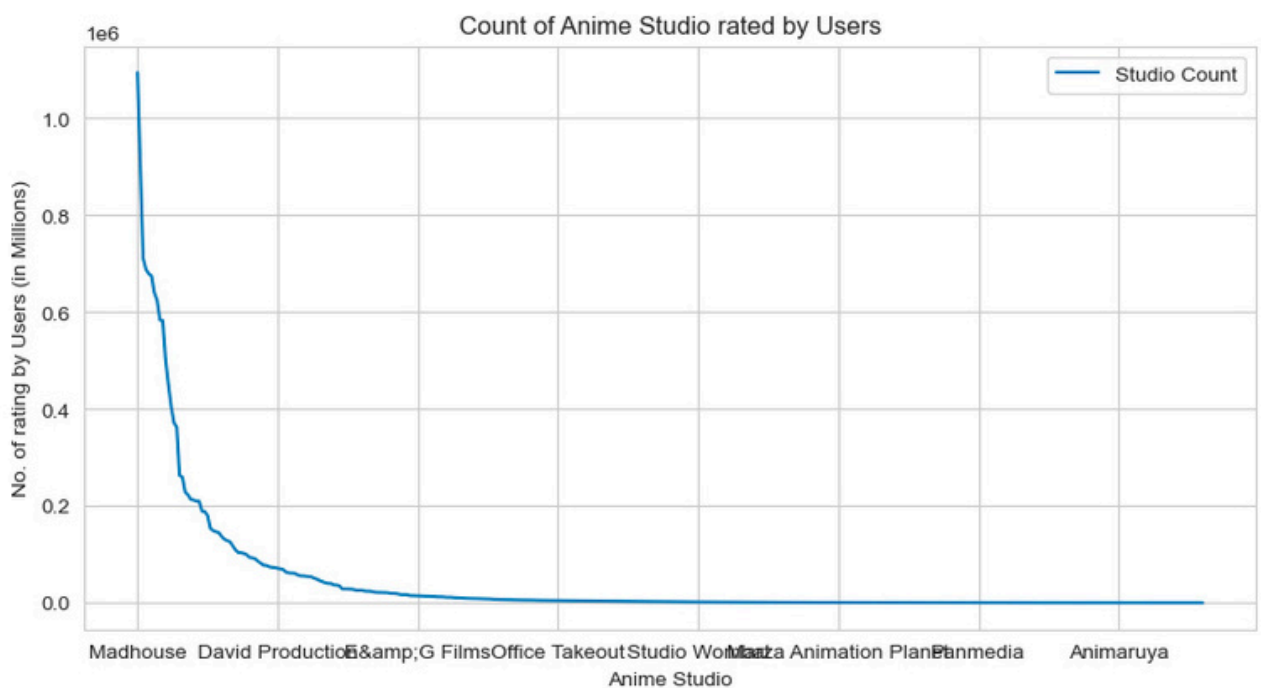


For 12 episodes , max rating is about 3.5

Analysis of Rating on Genres in Anime:



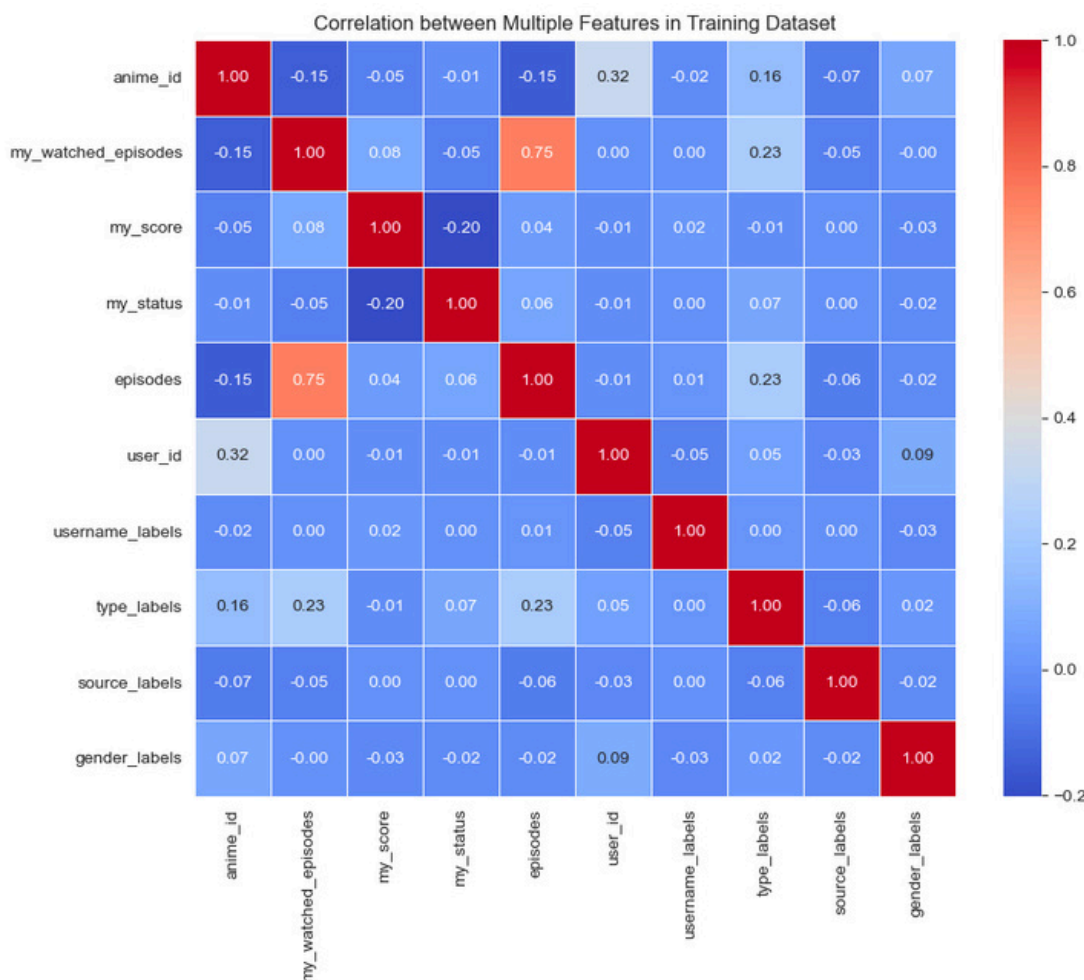
Analysis of Rating on Anime Studios: Various animes with more than one studio.



Results/Analysis/conclusion



Plotting Correlation between multiple Features:



Comparison of Content-Based and Collaborative Filtering Models

Model Type	Model Name	Methodology	RMSE	MAPE	Precision@5	Precision@10
Content-Based Filtering	Cosine Similarity	Bag of Words (BOW) and Cosine Similarity	2.4981	29.284	45.01%	44.62%
Collaborative Filtering (Memory-Based)	KNNBasic	Anime-Anime Similarity	1.648	16.288	51.8%	51.2%
	KNNBaseline	Anime-Anime Similarity	1.492	10.453	76.3%	74.9% (Best Precision)
	KNNWithMeans	Anime-Anime Similarity	1.642	10.452	63.50%	63.59%
Collaborative Filtering (Model-Based)	SVD	Latent Factor Decomposition	1.489	9.784	76.3%	74.6%
	Linear Regression	Feature Mapping	1.723	20.388	71.6%	70.3%
	SVR	Support Vector Regression	1.783	21.017	74.76%	72.95%
	LightGBM	Gradient Boost Decision Tree	1.591	16.837	72.81%	71.55%

Table 1: Comparison of Content-Based and Collaborative Filtering Models

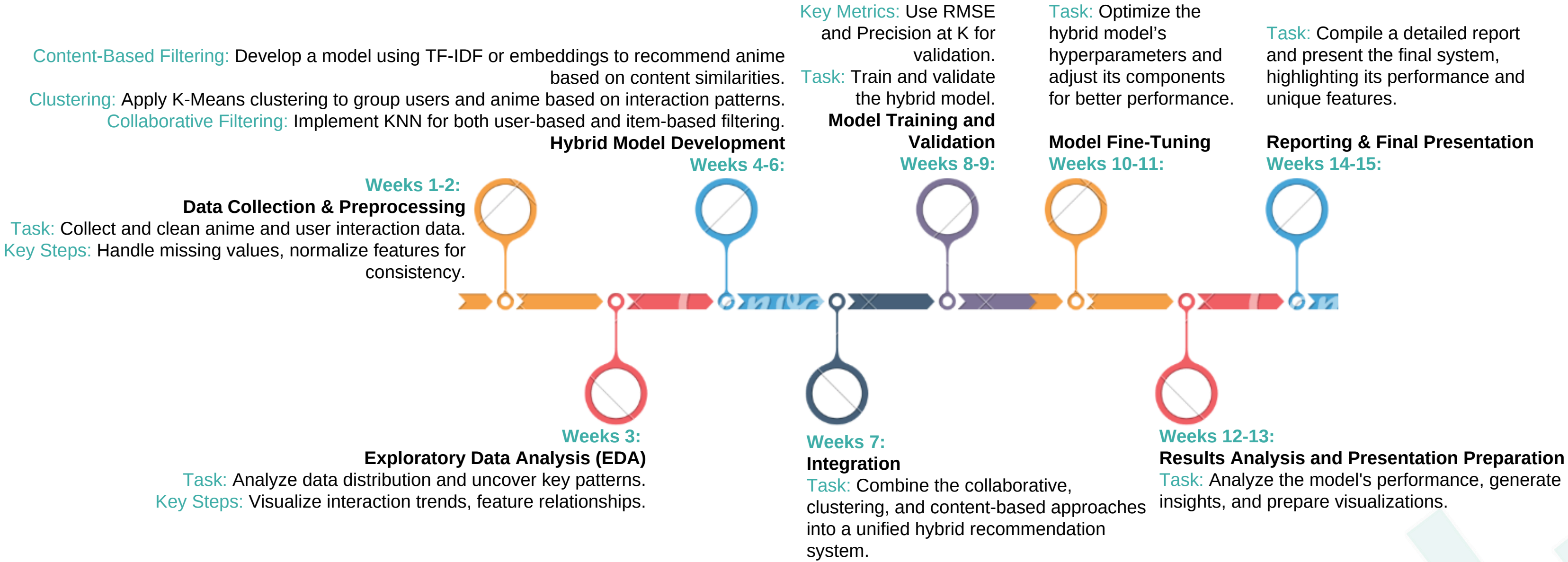
Conclusion:

- Precision: **KNNBaseline** (Precision@10: 74.9%) and **SVD** (Precision@10: 74.6%) excel in collaborative filtering, with SVD achieving the lowest RMSE (1.489).
- Content-Based vs. Collaborative Filtering: Collaborative filtering significantly outperforms content-based methods in both precision and error metrics, especially for sparse data.

Why KNNBaseline is Preferred?

It is easier to implement and interpret. Also, comparable precision and error to SVD with better handling of sparse datasets and more efficient for smaller or real-time applications. Hence, Collaborative filtering models, especially SVD and KNNBaseline, provide superior recommendations. While SVD uncovers deep relationships, KNNBaseline is often preferred for its simplicity and efficiency.

Timeline



Individual Team Members' Contributions

Contributions:

1. **Nishchal Gaur:** EDA and implementing ML models (Linear Regression, SVR).
2. **Yogesh Kumar:** Collaborative Filtering models and hybrid model integration.
3. **Sarthak Raj:** Performance evaluation, Front end development and report documentation.
4. **Niket Aggarwal:** Data cleaning and development of Content-Based Filtering.



Thank You